HexmodX

User Manual

BOSCH

Abstract

This document shall help the user of HexmodX to know how to work with the tool. The focus is set to enable the user to generate a modified hex file in a short time.

Table of Contents

Introduction

1.1 General

The tool HexmodX reads and modifies a Intel hex file and Motorola S19 file. This modification involves balancing checksum, compatibility modification, variant coding, and tuning protection. This tool is used by Software Build Environment, Calibration Engineers, INCA and Factory Release processes.

1.1.1 What is HexmodX?

HexmodX tool is designed for modular approach. Each functionality is separated into different packages. User can configure the loading of these package modules depending on the functionality required using a configuration file. This configuration file is in XML format. Each module package loading requires a XML configuration file. Each XML configuration file is validated against a schema defined by the tool.

1.1.2 What is XML?

XML (e**X**tensible **M**arkup **L**anguage) is a simple, very flexible text format derived from SGML. XML is a markup language for documents containing structured information. XML is mainly used for exchange of a wide variety of data between two parties.

To define the structure of XML-based configuration file, user need a “Schema” or a “DTD (Document Type Definition)”. In the header of each XML this schema name should be mentioned for validation purposes.

1.1.3 What is Schema?

“Schema” is a definition language used by XML for describing and constraining the content of XML documents. The “Schema” defines the grammar and structure of the document. User can define the structure, constraints and data types in Schema.

Relative Schema path:

Sample xml file for Schema Validation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\rules.xsd">
<HEXMOD>
    <MAINMOD>
        <LOADDLL>
            <DLL NAME="BaseIO" CFG="Input.xml"/>
            <DLL NAME="BaseIO" CFG="Output.xml"/>
        </LOADDLL>
    </MAINMOD>
<HEXMOD>
</CONF>
```

User must provide a space between www.rbin.com/mds-1 and .\schemas\rules.xsd in xsi:schemaLocation attribute. If the space is not provided, HexmodX will throw an error. User must provide
single dot (.) in ".\schemas\rules.xsd" then HexmodX tries to load the rules.xsd from the schema folder inside the HexmodX installation directory.

Configurable Schema path:

Sample xml file for Schema Validation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 C:\schemas\rules.xsd">
    <HEXMOD>
        <MAINMOD>
            <LOADDLL>
                <DLL NAME="BaseIO" CFG="Input.xml"/>
                <DLL NAME="BaseIO" CFG="Output.xml"/>
            </LOADDLL>
        </MAINMOD>
    <HEXMOD>
</CONF>
```

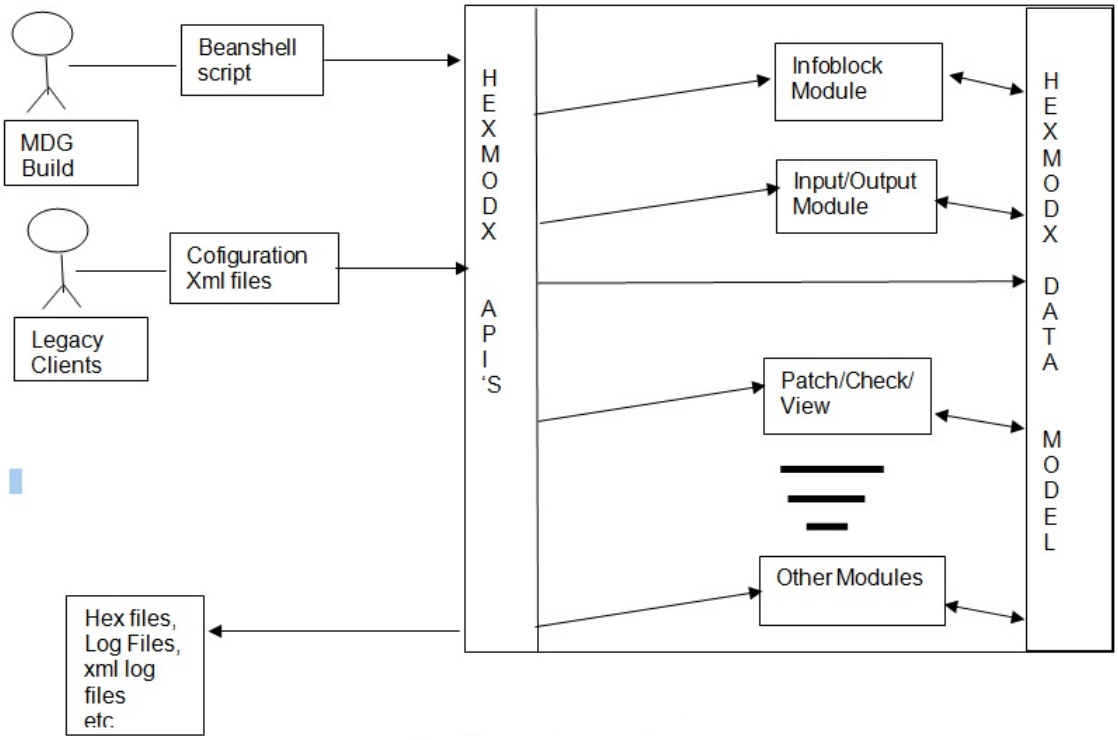
If the rules.xsd is not present in "C:\schemas\rules.xsd" then HexmodX tries to load the rules.xsd from the schema folder inside the HexmodX installation directory.

1. Overview

The tool HexmodX is designed to modify the hex files in Intel format and Motorola format. The modification involves merging of hex files, copying, moving and deleting data, balancing for checksum and compatibility, variant coding of data sets, tuning protection and others.

All the features are delivered in java archive (jar) format. The tool is driven by the rules set in the configuration files, which is in XML format.

The overview of the tool architecture is as shown below.



User can write their own modules, which can be completely independent. User modules can also access the data shared by modules provided by the HexmodX tool during its operation. This can be done by calling user modules using main configuration file, which is explained in later topics.

2.1 Functional Overview

HexmodX functionalities are delivered in java archive (.jar) format. HexmodX tool uses the configuration files in XML format to trigger the functionalities required. Each functionality are mentioned in the main configuration file (usually called as rules.xml) using XML tags sequentially. Hence modules are called one after the other. Its user's responsibility to provide the modules sequentially as per the functionality required.

The modules provided by the tool are
1. Error / Logging module
2. Base IO (Input/output) module
3. Infoblock module

4. ABM (Alternate Boot Module)
5. Checksum
6. Compression

The main module of the tool calls all these modules. This main module is entry point for the application. HexmodX tool has two entry points. One is an executable (called HexmodX.exe) and another is DLL for INCA (called hpt_HexmodX.dll). Both are used for same functionalities, however, INCA DLL is called by INCA.

2.1.1 Executable

HexmodX is a command line application which takes configuration file as command line parameter and uses it to read the configuration settings which includes the functions/operations the tool executes.

This application can be started by calling from dos prompt.

Calling HexmodX from dos prompt

Open the command prompt by doing following steps
Click Start button in windows → choose Run → type cmd → click OK
Command prompt opens
Type HexmodX along with configuration file as shown below

HexmodX also provides several other command line options for version check, help documentation. The following is a list of command line options which serve to trigger various operations in the application.

/help or /?	For basic command line help or usage of software
/h1 or /h2	For advanced command line help for advanced options (future usage)
/v	Software version, modules supported
/	Path and file name of configuration file /C:<Path\Config file name>
man	Manual entries for help documentation

All command line parameter must begin with “/”, except “man”

However, if no parameter is supplied, user shall be prompted dos box for the configuration/command file input as shown above.

2.1.2 INCA DLL

The interface is provided by the DLL will be accessed by INCA tool to invoke HexmodX functionalities.
On success TRUE is returned and on failure FALSE is returned.

The DLL name is HPT_HEXMODX.dll

The DLL interface is defined as

```
boolean HexPostTreatment
        (
            // path & fileName for hexfile
            unsigned char *hexPath,
            // path & fileName for a logfile
            unsigned char *logPath,
            // from asap2
            unsigned char *cpuType,
            // from asap2
            unsigned char *parameter
        );
```

The fourth parameter can contain the following values.

- CFG = <config file with path>

e.g. CFG = c:\program files\HexmodX\test\rules.xml

The configuration file can be provided to this call through the fourth parameter. **In this case, the first two parameters (hexPath and logPath) are not used**. This action is similar to executable.

- BLK_Start = <Infoblock address>

If the BLK_Start address is provided as fourth parameter, then initial two parameters are necessary to specify the hex file with path and log file with path.
e.g. BLK_Start = 0x80808080

This triggers the INFOBLK chain to be built based on the start address provided by fourth parameter. The checksum is calculated for the blocks appear in the block chain and compatibility is verified.

2.1.2 Main Module

This module is a main processing unit of HexmodX. This module is automatically loaded by either executable or INCA DLL using a configuration file.

This configuration file is used as the main configuration file for the HexmodX application to load the required functionalities.

Along with module names required to be loaded, this also defines the error handling settings and other general settings. User should be aware of the module names to load for the functionality required.

The sample configuration file is as shown below:

Note: Descriptions of each tags and attributes are explained in table below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\rules.xsd">
    <HEXMODX>
        <ERRMOD>
            <LOG FILE="HexmodX.log"/>
```

```
            <MSGLOG LVL="1"/>
            <MSGCONSOLE LVL="2"/>
            <PROGABORT LVL="3"/>
        </ERRMOD>
        <MAINMOD>
            <GENERAL>
                <DEF_RANGE START="0xA0000030" LEN="0xCCDD" FILLPATTERN="0xAA"/>
</GENERAL>
            <LOADDLL>
                <DLL NAME="BaseIO" CFG="HexmodX_Base_Input.xml"/>
                <DLL NAME="Infoblock" CFG="HexmodX_Infoblock.xml"/>
                <DLL NAME="BaseIO" CFG="HexmodX_Base_Output.xml"/>
            </LOADDLL>
        </MAINMOD>
    </HEXMODX>
</CONF>
```

Each XML file is validated against a schema. The main module XML file is validated against the schema “rules.xsd”, a schema file is provided by the HexmodX.

Elements	Children or Attributes	Description
CONF	**HEXMODX**	Root element of all the tags
HEXMODX	**ERRMOD, MAINMOD**	HEXMODX Tag is the basic tag for all the modules. This tag always appears after CONF tag. All HexmodX options are mentioned after this tag.
MAINMOD	**GENERAL, LOADDLL**	MAINMOD determines the sequence of operations that HexmodX executes. It has to contain LOADDLL tag, which specifies the DLLs to be loaded. GENERAL tag is optional.
GENERAL	**DEF_RANGE**	GENERAL tag contains valid RANGE which should be considered while processing. DEF_RANGE contains START, LEN, FILLPATTERN and FILLGAP attributes. START and LEN defines the RANGE of the valid region and FILLPATTERN specify the values to be considered for GAPs in the defined RANGE. FILLGAP is a Boolean attribute, if set to “true”, whole defined RANGE shall be filled with values specified by FILLPATTERN. By default, this value will be “false”.
	TRANSLATEADDRESS	TRANSLATEADDRESS tag contains a boolean attribute namely SEGMENTATOSEGMENT8. This tag defines the segmentA to segment8 conversion. This tag is used to bypass any operations on segmentA address to segmentt8 addresses. SegmentA addresses in the hex file are not modified instead any operations on these addresses are performed on its equivalent segement8 address. The default value of this element is true. That is this feature is enabled by default. The value of the attribute SEGMENTATOSEGMENT8 should be made false if the conversion from segmentA to segment8 is not required.
LOADDLL	**DLL**	LOADDLL tag shall have list of DLL tags, which shall contain information on the DLLs to be loaded.
DLL	**NAME, CFG**	DLL tag is used to specify the DLLs to be loaded. Using NAME attribute, DLL name can be specified. Using CFG

		attribute, the name of the configuration file can be specified.
FILELIST	**FILEIN** **DLL**	This FILELIST is used for batch processing of the input files supplied. This is followed by FILEIN keyword. Provide files to be processed in batch mode. Then define the DLL for the required functionality. FILELIST by default contains BASEIO DLL for reading and writing output. After processing all the DLLs, BASEIO DLL is automatically loaded for writing the modified file content. User need not specify the BASEIO DLL for read/write operation. For detailed explanation refer : Sec. 2.3 Batch Operation

Note:

Keyword “ERRMOD” define the log file name to put the log message levels.

This error module settings can be defined in all modules configuration file. If not defined, this main modules configuration settings are taken for error handling.

2.1.3 Base IO Module

Base IO module is used for basic Input/Output operations on Intel hex and Motorola s19 files. This module is used to read, modify and write them.

Either “INPUT” or “OUTPUT” can appear in side “BaseIO” tag. Base IO module needs to be called twice in any HexmodX operations through Main Module. i.e., Once for reading input files and Later for writing output files.

The INPUT tag is provided inside BASEIO tag to specify the settings for FILE read operations.

This has two input types.
- FILEIN (normal file read/merge operations)
- VDS (variant data coding operation)

This input tag also has additional tag for Copy, Move and Delete functionality called “MEM”.

The output tag is provided inside BASEIO tag to specify the settings for the FILE writing back operations.

This has two output types.
- FILEOUT (normal file read/merge operations)
- VDSBACK (variant data coding operation)

Elements	Children or Attributes	Description
BASEIO	**INPUT, OUTPUT**	**BASEIO** tag specifies the Input/Output operations. It can have either **INPUT** or **OUTPUT** tag.
INPUT	**FILEIN** **VDS**	**INPUT** tag specifies the file read operations. **FILEIN** tag will be used for normal reading and merging. **VDS** will be used for variant coding purposes. Either one of these elements can appear inside **INPUT** tag.
OUTPUT	**FILEOUT** **VDSBACK**	**OUTPUT** tag specifies the file write operations. **FILEOUT** tag will be used for normal writing of modified hex file contents.

		VDSBACK will be used for variant coding backwards purposes. Either one of these elements can appear inside **OUTPUT** tag.

Special data type is used for validating addresses provided in the XML file. It is "**exhexBinary**". This data type has particular size limit and pattern, which is as described below.

Characteristics	Values
MinLength	3
MaxLength	10
Pattern	[0]x[a-fA-F0-9]*

2.1.3.1 Input Operations

The Input operations involves reading Hex Files. Input operation is triggered by the keyword "FILEIN" inside INPUT XML tag. This keyword can appear more than once inside "INPUT" tag.

FILEIN keyword has TYPE, NAME, IGNORELINECKS, OVERWRITE, FROM, LEN attributes.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMODX>
        <BASEIO>
            <INPUT>
                <!-- IGNORELINECKS by default is false.-->
                <FILEIN TYPE="IHEX" NAME="file1.hex" OVERWRITE = "true"
MODE="BigEndian"/>
                <FILEIN TYPE="IHEX" NAME="file2.hex" IGNORELINECKS = "true"
MODE="BigEndian" />
                <FILEIN TYPE="IHEX" NAME=".\files\file3.hex" MODE="BigEndian"/>
            </INPUT>
        </BASEIO>
    </HEXMODX>
</CONF>
```

Multiple input files can be read one after the other. Multiple Files read operation results **in data merging**. File data is stored in ascending order of address. If any duplicate address is found while merging, application throws error. This can overridden by OVERWRITE attribute.

Elements	Children or Attributes	Description
FILEIN	**MEM**	FILEIN tag is used to specify the input file for reading purposes. This has different attributes for the This has MEM as child tag, which is used for data copy, move and delete operations.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19)

		Using this attribute, user can specify the file type provided for input.
	NAME	This is a mandatory attribute This attribute provides file name for input reading. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.
	FILLPATTERN	This is an optional attribute. This attribute fills the gaps with given fill pattern value for the complete input hex file.
	COPYFILE	This feature is included to copy a block of data from one file to another file. This is different from file merge in the sense, only the required block of data will copied from one file to another. But in File merge all the data from one file will be merged with another file. This is an optional field. This element can appear any number of times inside FILEIN tag. This element contains some attributes and a child element. The attributes are TYPE, NAME, IGNORELINECKS, OVERWRITE. TYPE attribute can take the type of the hex file which is the input (S19 or IHEX). NAME attribute has the path/Name of the hex file. IGNORELINECKS attribute is a Boolean value. If it is set to “true” Then the line check sums in the file mentioned in NAME attribute is ignored or if it is set to “false” the program aborts.

		OVERWRITE attribute is also a Boolean value. If it is set to “true” it means if there are any duplicate address inside the same file then overwrite that with the new value for that address. If it is set to “false” and if there is any duplicate address HexmodX aborts. There is a child element also present for COPYFILE element. The child element is **COPYBLOCK.** **COPYBLOCK** element is mandatory inside COPYFILE tag. This tag too can appear any number of times. The attributes for this element are FROM, LEN, TO, FILLGAPS, OVERWRITE. FROM attribute specifies the start address of the block to be copied from the file given inside COPYFILE tag. LEN gives the length of the block. TO gives the destination address of the block to be copied. If the destination address is not present in the original file then this address is created and inserted. FILLGAPS attribute specifies the value to be filled if there are any gaps in the block. OVERWRITE is again a Boolean value which if set to “true” the block is overwritten with the old value. If OVERWRITE flag is not set to true, HexmodX will NOT abort but continue and dose not copy the block to the destination.
	OVERWRITE	This is an optional boolean attribute with default value “false” Overwrite option is used while merging multiple files read during INPUT operation. If this option is set to “true”, and if any duplicate address record found with existing data and data while reading subsequent files.
	MODE	This attribute defines the endianess of hex file. Either LittleEndian or BigEndian can only be specified for endianess. It is a mandatory attribute.
FILLGAPS	**START** **LEN** **FILLPATTERN**	FILLGAPS element appear after all FILEIN keywords inside INPUT keyword.

		This contains START, LEN and FILLPATTERN attributes. The GAPS in the regions defined by this will be fill-up using FILLPATTERN after reading all the input files. This is an optional element, which will be ignored, if DEF_RANGE is provided in the “rules.xml”.
FILLRANGE	**START** **LEN** **FILLPATTERN**	FILLRANGE element appears after the FILEIN keywords, inside INPUT element. This element can occur either after the FILLGAPS element or before the FILLGAPS element. FILLRANGE is used to fill a block of data in the hex file with a given pattern. This element can occur any number of times inside the INPUT tag. This contains START, LEN and FILLPATTERN attributes. START attribute gives the start address from which the filling begins. LEN gives the length of the block to fill. FILLPATTERN attribute specifies the pattern with which the block has to be filled. This attribute is optional. If not mentioned, by default “0xFF” value is filled in the range. FILLRANGE is an optional element inside the INPUT tag.
	INPUTRANGES	This feature is included to merge two input HEX files based on given data ranges. The attributes are STARTADDRESS, ENDADDRESS, OVERWRITE, FILLGAPS. STARTADDRESS - Start address of the range which needs to be merged with another hex file. ENDADDRESS - End address of the range. FILLGAPS - attribute specifies the value to be filled if there are any gaps in the given range. OVERWRITE - is a Boolean value if set to “true” it will overwrite the data present at the duplicate address in the previous ranges or previous HEX file.

2.1.3.2 Output Operations

The Output operations involves writing Hex Files. Output operation is triggered by keyword "FILEOUT" inside OUTPUT XML tag. This keyword can appear more than once inside "OUTPUT".

FILEOUT

This XML tag specifies the output operations to be done by HexmodX application. This has TYPE, NAME, FROM, LEN and LYT attributes.

FROM and LEN attributes can be used to write the output files for a range specified.

The sample XML input file is as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMODX>
        <BASEIO>
            <OUTPUT>
                <FILEOUT TYPE="IHEX" NAME="file1.hex" />
                <FILEOUT TYPE="IHEX" NAME="file2.hex" />
                <FILEOUT TYPE="S19"  NAME=".\files\file3.hex"/>
            </OUTPUT>
        </BASEIO>
    </HEXMODX>
</CONF>
```

Elements	Children or Attributes	Description
FILEOUT		FILEOUT tag is used to write the modified hex file contents into a file specified. This tag is a child of OUTPUT tag.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.
	NAME	This is a mandatory attribute This attribute provides file name for writing output file. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	FROM	This attribute is used to provide the start address for the writing the data over the range. Data type is **exhexBinary.**
	LEN	This attribute is used for the length of the range for writing the data into output file. Data type is **exhexBinary.**
	LYT	This attribute is used to configure the layout (byte length) of each data records, while writing the output file. This can be configured from "0x00" to "0xFF".
	IGNORE3OR5	This is a boolean attribute, used to ignore records of type "03" or "05" while writing the output files.

	FORMAT	This attribute is optional and applies only when TYPE attribute is “S19”. The possible values of this attribute are S2 and S3. The value of this attribute specifies the format of the S19 file while writing the output file. S3 format means that the S19 file contains 4 byte address and S2 format means S19 file contains 3 byte address. By default the value of this is S3.
	SORT	This attribute is optional and accepts Boolean values. When the value is true, the range of data to be written in output file is sorted on the basis of address and if the value is false, the ranges are written the order which is given in the RANGE element.
	FILLPATTERN	This is an optional attribute. This attribute fills the gaps with given fill pattern value for the specified output hex file.
	RWG	This is an optional attribute. RWG file is generated when RWG attribute value is **ENABLED.** HexmodX aborts if different value is specified.

2.1.3.2.1 RWG file generation

To generate RWG file, specify the RWG xml configuration file in **RWG_FILE** attribute of INPUT element and set value “**ENABLED**” in RWG attribute of FILEOUT element. HexmodX aborts if different value is specified.

Sample xml file is as below,

Input xml file:

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1
c:\schemas\baseio.xsd">
    <HEXMOD>
        <BASEIO>
            <INPUT RWG_FILE="RWG_parameters.xml">
                <FILEIN TYPE="IHEX" NAME="input.hex" IGNORELINECKS= "true"
MODE="LittleEndian" />
            </INPUT>
        </BASEIO>
    <HEXMOD>
</CONF>
```

Output xml file:

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1
c:\schemas\baseio.xsd">
    <HEXMOD>
        <BASEIO>
            <OUTPUT>
                <FILEOUT TYPE="BIN"
NAME="./../../HEXMODX/_out/medc18_out_HexmodX.rwg" RWG="ENABLED" />
```

```
            </OUTPUT>
        </BASEIO>
    <HEXMOD>
</CONF>
```

2.1.3.3 Append Features

APPEND features is provided to merge several hex range data into single hex file. This feature is provided by FILEOUT element used in the OUTPUT operation. FILEOUT has a child element “RANGE” defined for this operation. This element shall appear multiple times to define the more ranges. “RANGE” feature uses the file “NAME” and “TYPE” of the output element “FILEOUT”. So, further defining the file name is not required. “FROM” and “LEN” attributes of “FILEOUT” is not required to define during the merging of hex file data over ranges defined by “RANGE” element.

Elements	Children or Attributes	Description
RANGE	**FROM, LEN**	This element is used to define the RANGE for merging hex file data into single hex file. FROM and LEN attributes are provided to define the range for merging. These attributes hold the values of type “**exhexBinary**”.

Sample output XML file is as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"  xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMOD>
        <BASEIO>
            <OUTPUT>
                <FILEOUT TYPE="IHEX" NAME="file1.hex">
                    <RANGE FROM=”0x80808000” LEN=”0x400” />
                    <RANGE FROM=”0x809C0000” LEN=”0x8000” />
                <FILEOUT/>
            </OUTPUT>
        </BASEIO>
    <HEXMOD>
</CONF>
```

2.1.3.4 MEM Operations

MEM keyword is used for Copying, Moving and Deleting the range of data specified. This element is a child of FILEIN. If any data copy, move or delete operation is required, MEM keyword shall be used immediately as sub element inside FILEIN keyword.

Sample XML file is as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMODX>
        <BASEIO>
            <INPUT>
                <!-- IGNORELINECKS by default is false.-->
                <FILEIN TYPE="IHEX" NAME="file1.hex" OVERWRITE = "true"/>
                <FILEIN TYPE="IHEX" NAME="file2.hex" IGNORELINECKS = "true" />
                <FILEIN TYPE="IHEX" NAME=".\files\file3.hex">
                    <MEM ACTION="COPY" FROM="0x80807F74" LEN="0x8B"
TO="0x80807F74" OVERWRITE="true" FILLGAPS="0xC3"/>
                    <MEM ACTION="MOVE" FROM="0x80807F74" LEN="0x8B"
TO="0x80807F74"/>
                    <MEM ACTION="DEL" FROM="0x80807F74" LEN="0x8B"/>
                </FILEIN>
            </INPUT>
        </BASEIO>
    </HEXMODX>
</CONF>
```

Elements	Children or Attributes	Description
MEM		**MEM** keyword is used for Copying, Moving and Deleting the range of data specified. This element is a child of FILEIN. If any data copy, move or delete operation is required, MEM keyword shall be used immediately as sub element inside FILEIN keyword.
	ACTION	This attribute provides the trigger to do the specified operations. The values can be "COPY" or "MOVE" or "DEL". "COPY" value triggers the copy action from the address specified by "FROM" to the location mentioned by "TO" attribute for the range specified by "LEN" "MOVE" value triggers the copy action from the address specified by "FROM" to the location mentioned by "TO" attribute for the range specified by "LEN". This action deletes the source data. "DEL" value triggers the deletion from the address specified by "FROM" for the length of "LEN".
	FROM	This attribute is used to provide the start address for the source data. Data type is **exhexBinary.**
	LEN	This attribute is used for the length of the range for writing the data into output file. Data type is **exhexBinary.**
	TO	This attribute is used to provide the destination address for the writing the data. Data type is **exhexBinary.**
	OVERWRITE	This is an optional boolean attribute with default value "false" If this option is set to "true", and if any duplicate address record found during any move or copy operation, data will be overwritten.

	FILLGAPS	This is an optional boolean attribute with default value “0xFF” This attribute specifies the values to be considered for GAPS.

2.1.3.5 Variant Coding

Variant coding is done using HexmodX by triggering command “VDSIN”. VDSIN is a child tag under INPUT tag. This takes several parameters to produce variant coding operation. They are as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMODX>
        <BASEIO>
            <INPUT>
                <VDS VDSLOG="VDSLOG.txt" VCTRTABLELOG="VctrTableLog.txt">
                    <VDSIN TYPE="IHEX" NAME="refvariant_0.hex"
VCTRINFOBLKADDRESS="0x880058" INFOBLOCKADDRESS="0xA0000008"/>
                    <VDSIN TYPE="IHEX" NAME="refvariant_1.hex"
VCTRINFOBLKADDRESS="0x880058" INFOBLOCKADDRESS="0xA0000008"
VCTRINFOLOG="refvar1info.log"/>
                </VDS>
            </INPUT>
        </BASEIO>
    </HEXMODX>
</CONF>
```

VDSIN tag is used for Variant Data coding operations. This operation is specified by its parent VDS tag under INPUT keyword.

VDSIN keyword has TYPE, NAME, IGNORELINECKS, OVERWRITE, INFOBLOCKADDRESS, VCTRINFOBLKADDRESS, , VCTRINFOLOG attributes.

Elements	**Children or Attributes**	**Description**
VDS	**VDSIN**	VDS tag is used for Variant coding purposes, which has VDSIN tag as mandatory child tag.
	VDSLOG	**VDSLOG** which takes name of a file to which the details of the data set sizes will be written.
	VCTRTABLELOG	**VCTRTABLELOG** takes a filename as a input to which vector table contents of variant coded file is written.
	VCTRINFOLOG	**VCTRINFOLOG** takes the file name as input to which the vector Infoblock table contents of variant coded file is written.
VDSIN	**MEM**	VDSIN triggers the variant coding operations for files supplied using NAME attribute. This has child element MEM, so that MEM operations can also be carried out once file is read before variant coding.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.

	NAME	This is a mandatory attribute This attribute provides file name for input reading. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.

During Variant Coding operation, three different information files will be written based on the request by in the XML file.

VDSLOG report will be written to an output file specified by the configuration setting “**VDSLOG**” Total free space in Data Set of Variant Coded file will be calculated and displayed in the report.

```
[HexmodX]    [7.1.0]


Copyright (c) Robert Bosch GmbH Ltd.

#########################################################################
#  Data sizes of variant files in VDS                                   #
#########################################################################
Files                Size(in bytes)      %Occupation          %Free
-------------------------------------------------------------------------
refvariant_0.hex              48                 4.17
refvariant_1.hex             100                 8.68
refvariant_2.hex              12                 1.04
refvariant_3.hex             160                13.89
refvariant_4.hex              12                 1.04
refvariant_5.hex              12                 1.04
refvariant_6.hex             272                23.61
-------------------------------------------------------------------------
Total                        616                53.47          46.53
-------------------------------------------------------------------------
```

VCTRTABLELOG command will write the contents of the vector table into a log file. This will be sometimes necessary for the user/developer to check the location of vector table and its contents.

```
[HexmodX]    [7.1.0]

Copyright (c) Robert Bosch GmbH Ltd.

#########################################################################
#  Vector Table Information                                             #
#########################################################################
```

```
AREA: <DS> at 0x800058
[START]
0x8e064e
0x8e0656
……..
[END]
AREA: <VDS_1> at 0x880084
[START]
0x882dac
0x8e0656
0x8e065c
……..
[END]
Area: %VDS_%n% %ADDRESS%
[START]
0x8e064e
0x8e0656
0x8e065c
……..
[END]
```

VCTRINFOLOG command will write the contents of the vector Infoblock details into a log file. This will be sometimes necessary for the user/developer to check the location of vector Infoblock and its contents.

```
[HexmodX]   [7.1.0]


Copyright (c) Robert Bosch GmbH Ltd.

#########################################################################
#  Vector Info Table Details                                            #
#########################################################################
Vector Table            : 0x800058
CheckSumInfoBlock_1     : 0
DS Vector Table         : 0x800058
DS Vector Table         : 0x8004d8
VDS Vector Table        : 0x880084
VDS Vector Table        : 0x882dac
VDS Data Start          : 0x882dac
VDS Data End            : 0x8dff74
Data Skip Start         : 0x1000000
Data Skip End           : 0
Vector Entries          : 8
```

2.1.3.6 Variant Coding Backwards

Variant Coding Backwards is operation to get back original data sets from variant coded file. Original data sets will be extracted from Variant coded file (VDS Area), and each individual file will be created.

VDSBACK keyword has TYPE, NAME, VDSLOG attributes.

Elements	Children or Attributes	Description
VDSBACK	-	VDSBACK is used for variant coding backwards. I.e., retrieval of variants from master file. This attribute is a child element of OUTPUT tag.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.
	NAME	This is a mandatory attribute. This attribute provides file name for input reading. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	VDSLOG	This is an optional attribute. VDSLOG which takes name of a file to which the details of the data set sizes will be written.

2.1.3.7 SegmentA to Segment8 Transition during Variant Coding

While performing variant coding, if the segmentA addresses are present in the DS vector table of master hex file then the segmentA addresses have to be retained in the vector table of VDS, though after translation, the data to be compared is always fetched from segment 8 addresses.

DS master vector table

0xA0001234	**- - - - - - - - - - - - -**

DS variant vector table

0xA0001234	**- - - - - - - - - - - - -**

VDS vector table

0xA0001234	**- - - - - - - - - - - - -**

Retained as segment A address in VDS section

2.1.4 Infoblock Module

Infoblock module is required to perform the modification of infoblocks inside MDG1 and MDG2 software. This Infoblock module performs linking of each blocks, modify blocks for checksum, compatibility, SWID, TTNr update etc.

Infoblock operations for Independent blocks (which are not part of infoblock link chain) can also be done in the same configuration file.

Currently, only BLKLINK and WRITECKS functionalities are supported for MDG2.

XML keywords for Infoblock operations are provided inside XML tag "**IBMOD**". They are

- Building & Linking Infoblock IBSTART or IBBUILD
- Log Infoblock before validation / modification LOGBEFORE
- Validate or Balancing Infoblock VALIDATE
- Log Infoblock after validation / modification LOGAFTER

Sample XML file is as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\baseio.xsd">
    <HEXMODX>
        <IBMOD>
            <!--IBSTART CFG="ibdetail.xml"/-->
            <IBBUILD>
<BLKLINK START="0x80808008" NXTBLK="0x80810000"/>
                <BLKLINK START="0x80810000" NXTBLK="0x80820000"/>
                <BLKLINK START="0x80820000" NXTBLK="0x0"/>
            </IBBUILD>
            <LOGBEFORE>
                <IBLOG FILE="ibbefore.txt"/>
                <CKLOG FILE="cksbefore.txt"/>
            </LOGBEFORE>
            <VALIDATE>
                <SWID BLKADDR="0x80808008" VAL="12345678"/>
                <SWTTNR BLKADDR="0x80808008" VAL ="1234567890"/>
                <ENDPATTERN FUNCTION="TEST"/>
                <CHECKSUM>
                    <WRITECKS FUNCTION="SET"/>
                    <COMPATIBILITY FUNCTION="SET">
                        <IB BLKADDR ="0x80820000" BLKCKS="0"
DEPBLKADDR="0x80810000" DEPCKS="0"/>
                    </COMPATIBILITY>
                </CHECKSUM>
            </VALIDATE>
            <LOGAFTER>
                <IBLOG FILE="ibafter.txt"/>
                <CKLOG FILE="cksafter.txt"/>
            </LOGAFTER>
        </IBMOD>
    </HEXMODX>
</CONF>
```

To include independent blocks in same configuration xml file, introduce another IBMOD element in xml file. For example,

```
<HEXMOD>
 <IBMOD>
        <IBBUILD>
              <BLKBUILD START="0x08FC0020" />
       </IBBUILD>
</IBMOD>

 <IBMOD>
        <IBBUILD>
          <BLKBUILD START="0x0060C100" />
        </IBBUILD>
</IBMOD>

  <IBMOD>
        <IBBUILD>
               <BLKBUILD START="0x00610100" />
         </IBBUILD>
  </IBMOD>
 <HEXMOD>
```

Elements	Children or Attributes	Description
IBMOD	**IBBUILD** **IBSTART** **LOGBEFORE** **VALIDATE** **LOGAFTER**	Using IBMOD tag, Infoblock processing operations are done. It has separate tag for each functionality. They are described in sections below.
	MDGTYPE	It specifies the Infoblock structure of MDG platform hex file. It takes **"MDG1"** and **"MDG2"** string values.

2.1.4.1 Building and Linking Infoblock

Building and Linking Infoblock can be done using two elements provided inside **IBMOD** tag. They are **IBBUILD** and **IBSTART**.

IBBUILD has two child elements, they are "BLKBUILD" and "BLKLINK".

BLKLINK: Linking the Infoblock depends on the start address provided in the BLKLINK element. This start address can be provided by attribute START. Linking Infoblock depends on the NXTBLK attribute value.

HexmodX application will pickup the START value and updates the Next Block field of the Infoblock to the value specified by NXTBLK.

Caution: User must be careful while choosing the start address of the Infoblock. HexmodX application updates NXTBLK value at offset 0x8 from START address location.

Attribute DIVISION="GSTC" is introduced in **IBBUILD** for GSTC division to handle its hex file. If DIVISION attribute with value GSTC is present, HexmodX links block address i.e. NXTBLOCK attribute value which is not present in input hex file with START attribute block address.

Sample Build Infoblock is as shown below.

```
<IBBUILD>
<BLKLINK START="0x80808008" NXTBLK="0x80810000"/>
    <BLKLINK START="0x80810000" NXTBLK="0x80820000"/>
    <BLKLINK START="0x80820000" NXTBLK="0x0"/>
</IBBUILD>
```

Elements	Children or Attributes	Description
IBBUILD	**BLKLINK** **BLKBUILD**	**IBBUILD** tag has two child elements. They can be used for only linking and building purposes. **BLKLINK** – links Infoblock which appears in the list. **BLKBUILD** – builds the already linked Infoblock for processing.
	DIVISION	It is an optional attribute and takes string values. By default, HexmodX supports DGS hex file. If the value is "GSTC", HexmodX behaves accordingly.
BLKLINK	**START**	Infoblock linking can be done between the start addresses and next block addresses of the subsequent Infoblock. Using START attribute, user can specify the Infoblock address to which next Infoblock to be linked.
	NXTBLK	Using NXTBLK attribute value, present Infoblock's next block address field will be updated. NXTBLK address field will be at an offset of 8 bytes from START address.
BLKBUILD	**START**	Already linked Infoblock should be the input. Using START attribute, tool will build the Infoblock structure by traversing through the Infoblock chain for further processing.

2.1.4.2 Infoblock Log Information Before and After update

Infoblock contents can be written to normal ASCII format files for verification purposes. This can be done BEFORE or AFTER updating Infoblocks.

LOGBEFORE tag can be used to trigger this operation before updating or validating Infoblock.

```
<LOGBEFORE>
<IBLOG FILE="ibbefore.txt"/>
    <CKLOG FILE="cksbefore.txt"/>
</LOGBEFORE>
```

Or

```
<LOGBEFORE NUTZDATEN_CHECKSUM ="ALL"/>
<IBLOG FILE="ibbefore.txt"/>
    <CKLOG FILE="cksbefore.txt"/>
</LOGBEFORE>
```

LOGAFTER tag can be used to trigger this operation after updating or validating Infoblock.

```
<LOGAFTER>
    <IBLOG FILE="ibafter.txt"/>
    <CKLOG FILE="cksafter.txt"/>
</LOGAFTER>
```

Or

```
<LOGAFTER NUTZDATEN_CHECKSUM ="ALL">
    <IBLOG FILE="ibafter.txt"/>
    <CKLOG FILE="cksafter.txt"/>
</LOGAFTER>
```

Elements	Children or Attributes	Description
LOGBEFORE **LOGAFTER**		**LOGBEFORE** is used to write the Infoblock contents to an ASCII file specified by attribute FILE before processing Infoblock. **LOGAFTER** is used to write the Infoblock contents to an ASCII formatted file specified by attribute FILE after processing Infoblock.
	IBLOG	**IBLOG** is used for only Infoblock details
	CKLOG	**CKLOG** is used for Checksum block details
	FILE	**FILE** attribute used for specifying the file name for log purposes. The file extension will be TXT. Two file will be generated with Infoblock and Checksumblock information. For IBLOG, The file extension can be XML also. If it's XML, all information will be written in XML format. In this xml file, checksum block information also written with the Infoblock information.
	NUTZDATEN_CHECKSUM	**NUTZDATEN_CHECKSUM** is an optional attribute for LOGBEFORE and LOGAFTER element that calculates SHA256 checksum for the block. This attribute can only accept value **ALL.** The calculated checksum value is written into the file specified in the IBLOG element in LOGBEFORE and LAGAFTER element.

2.1.4.3 Logging Information of Blocks which are marked as OTP.

Blocks are said to be marked as OTP (One Time Programmable) if the **23rd bit of BLKID** is high.

This feature enables the user to detect whether blocks are marked as OTP are not. If the blocks are marked as OTP then it logs information about the block like STARTADDRESS,ENDADDRESS and Nutzdaten Checksum in the txt file or xml file specified in the <IBLOG> tag of InfoBlk.xml as shown below. If NUTZDATEN_CHECKSUM attribute is specified in the LOGBEFORE then Nutzdaten Checksum will be calculated for all the blocks.

```
<LOGBEFORE>
        <IBLOG FILE="ibbefore.txt"/>
</LOGBEFORE>
```

Or

```
<LOGBEFORE>
        <IBLOG FILE="ibbefore.xml"/>
</LOGBEFORE>
```

2.1.4.4 Validating Infoblock

Validation of Infoblock is triggered by VALIDATE keyword in the XML configuration file for Infoblock handling. This validation involves writing SWID, SWTTNR, Balancing for Checksum and Compatibility.

Validation needs to be done for SET or TEST operations on various features. SET operation will update the values. TEST operation will check the values present for the actual value.

A sample XML file is as shown below.

```
<VALIDATE>
    <SWID VAL="12345678" FUNCTION = "SET"/>
    <SWID BLKADDR ="0x80808008" VAL="12345678" FUNCTION = "SET"/>
    <SWTTNR VAL="1234567890" FUNCTION = "SET"/>
    <SWTTNR BLKADDR ="0x80808008" VAL="1234567890" FUNCTION = "SET"/>
    <SWTTNR BLKADDR ="0x80808008" VAL="1234567890" FUNCTION = "TEST" FILE =
"SwttnrTEST.xml" />
    <ENDPATTERN FUNCTION="TEST"/>
    <CHECKSUM>
        <WRITECKS FUNCTION="SET"/>
        <COMPATIBILITY FUNCTION="SET">
    <IB BLKADDR ="0x80820000" BLKCKS="0" DEPBLKADDR="0x80810000" DEPCKS="0"/>
    <IB BLKADDR ="0x80B00000" BLKCKS="0" DEPBLKADDR="0x80820000" DEPCKS="0"/>
    <IB BLKADDR ="0x80C00000" BLKCKS="0" DEPBLKADDR="0x80B00000" DEPCKS="0"/>
        </COMPATIBILITY>
    </CHECKSUM>
</VALIDATE>
```

SWTTNR :

Elements	Children or Attributes	Description
VALIDATE		
	SWID	SWID tag is used to SET/TEST values present in the hex file and value provided along with VAL attribute. FUNCTION attributes triggers the SET/TEST functionality. User can either specify the specific Infoblock to be updated using BLK attribute.
	SWTTNR	SWTTNR tag is used to SET/TEST values present in the hex file and value provided along with VAL attribute. FUNCTION attributes triggers the SET/TEST functionality. User can either specify the specific Infoblock to be SET/TEST using BLK attribute. User can also generate a XML file for the function TEST.
	ENDPATTERN	ENDPATTERN tag is used to SET/TEST values present in the hex file at ENDPATTERN field. FUNCTION attributes triggers the SET/TEST functionality. SET functionality will set the ENDPATTERN field to "0xDEADBEEF" and TEST functionality will check whether ENDPATTERN field has "0xDEADBEEF" value or not. Any mismatch will trigger the Program Abort.

Writing SWID and SWTTNR

Elements	Children or Attributes	Description
SWID, SWTTNR		These elements are used to write/check the **SWID** and **SWTTNR** inside each Infoblock.
	VAL	This attribute provides an 8 or 16 character value for **SWID** depending on the **TYPE** attribute and 10 or 20 character value for **SWTTNR** depending on the **TYPE** attribute, to be **SET** or **TEST** for the functionality requested by **FUNCTION** attribute.
	BLKADDR	This is an optional field. **BLKADDR** attribute will provide the block address of Infoblock for which specified operation should be performed. If not given, all Infoblocks that appear in the Infoblock chain will be processed.
	FUNCTION	**FUNCTION** attribute will have either **TEST**/**SET** value. If **TEST** is used, values at respective location will be checked against value provided using **VAL** attribute. Failure in the match will result in an warning. **SET** will update the value given in **VAL** attribute in the specific locations of **SWID** and **SWTTNR** of the respective Infoblock.
	TYPE	This is an optional field. If given, it should either be **ASCII** or **HEX**. This attribute indicates whether the value provided in **VAL** is ASCII value or HEX value. If the value is hex the **VAL** attribute must start with 0x.
	FILE	**FILE** generates file (Only XML file is supported) for the SWTTNR operation. It is supported only for **SWTTNR TEST** function.

Balancing Infoblock

Balancing Infoblock is done by updating Checksum and Compatibility. These values can be tested or modified depending on the values TEST/SET.

CHECKSUM element triggers SET/TEST of Checksum & Compatibility of Infoblock.

WRITECKS does the Checksum calculation and COMPATIBILITY does the calculation of Compatibility.

If Only WRITECKS tag is provided, HexmodX "de-activate" the COMPATIBILITY i.e., it resets all adBlkID field of every Infoblock.

If both tags are provided, WRITECKS & COMPATIBILITY are handled for every Infoblock based on the Dependency link provided in the COMPATIBILITY list.

Compatibility ID and Compatibility Ref values can be patched directly for a given block directly in hex file using **BLOCK** element present under **COMPATIBILITY** element. Type of value can be IHEX or ASCII.

A sample XML file is as shown below.

```
<VALIDATE>
    <CHECKSUM>
        <WRITECKS FUNCTION="SET"/>
        <COMPATIBILITY FUNCTION="SET">
    <IB BLKADDR ="0x80820000" BLKCKS="0" DEPBLKADDR="0x80810000" DEPCKS="0"/>
    <IB BLKADDR ="0x80B00000" BLKCKS="0" DEPBLKADDR="0x80820000" DEPCKS="0"/>
    <IB BLKADDR ="0x80C00000" BLKCKS="0" DEPBLKADDR="0x80B00000" DEPCKS="0"/>
        </COMPATIBILITY>
    </CHECKSUM>
</VALIDATE>

<VALIDATE>
    <CHECKSUM>
        <WRITECKS FUNCTION="SET"/>
        <COMPATIBILITY FUNCTION="SET">
    <BLOCK START="0x80820000" COMPID="0x12345678AABBCCDD"
COMPREF="11223344ABCDEF12" TYPE="IHEX" />
        </COMPATIBILITY>
    </CHECKSUM>
</VALIDATE>
```

Elements	Children or Attributes	Description
CHECKSUM	**WRITECKS** **COMPATIBILITY** **WARNOVERWRITE**	**CHECKSUM** element triggers SET/TEST of Checksum & Compatibility of Infoblock. **WARNOVERWRITE** is a boolean attribute of CHECKSUM element. This has default value as “true”, which means, if checksum fields don’t contain “0xAFAFAFAF”, warning message will be shown. If user wants to suppress this warning, this attribute should contain “false” value.
	FUNCTION	SET function will overwrite the existing values. If existing values are not “0xAFAFAFAF”, warning message is displayed to user. TEST function will also calculate the checksum and verifies the new checksum value with the existing checksum value. Any mismatch will trigger the program abort.
WRITECKS		This tag triggers the checksum functionality for all Infoblock and Checksum Infoblock. Checksum calculation will be done based on the ascending order of end addresses of each Checksum Infoblock.

COMPATIBILITY		This tag triggers the Compatibility functionality for all Infoblock and Checksum Infoblock. SET function triggers the update of the Compatibility files based on the dependency link provided in the XML settings BLKADDR and DEPBLKADDR Compatibility Checksum calculation depends on the Infoblock (BLKADDR) and Dependency Infoblock (DEPBLKADDR). It also depends on the Checksum blocks specified by (BLKCKS) and (DEPCKS).
	IB	IB is a child element of COMPATIBILITY, which represents Infoblock address.
	BLKADDR	Specifies the Infoblock name from which compatibility to be checked.
	BLKCKS	Specifies the Checksum block number of the Infoblock from which compatibility to be checked.
	DEPBBLKADDR	Specifies the Infoblock over which main block (BLK) has dependency.
	DEPCKS	Specifies the Checksum block number over which main block (BLK) has dependency.
	BLOCK	This element represents an Infoblock in hex file.
	START	Specifies the Infoblock address in hex file.
	COMPID	Specifies the Compatibility ID value. Length of value is 0x8 bytes. Warning will be thrown if it is not 8 byte.
	COMPREF	Specifies the Compatibility Ref value. Length of value is 0x8 bytes. Warning will be thrown if it is not 8 byte.
	TYPE	Value can be IHEX or ASCII.

2.1.4.5 Deletion of a Infoblock

HexmodX has a new feature to delete an Infoblock, including all the check sum blocks, using one command. Till date any deletion of memory blocks from hex file was done using MEM operation in input.xml. This special feature to delete a complete Infoblock with all its checksum blocks in one command is provided in Infoblock.xml. The details of the configuration file are given below.

To delete for MDG2 infoblock, specify **MDGTYPE** attribute with value “**MDG2**” in <IBBUILD> element.

Sample file of the same is as follows

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="www.rbin.com/mds-1
M:\hexmod_paj3kor\hexmod17\schemas\infoblock.xsd">
    <!-- This is the input to the infoblock dll -->
    <HEXMOD>
        <IBMOD>
            <IBBUILD>
                <BLKLINK START="0x80820000" NXTBLK="0x809C0000"/>
                <BLKLINK START="0x80810000" NXTBLK="0x80820000"/>
                <BLKLINK START="0x809C0000" NXTBLK="0x0"/>
                <BLKLINK START="0x80808000" NXTBLK="0x80810000"/>
            </IBBUILD>
            <LOGBEFORE>
                <IBLOG FILE=".\TC065\01\out\ibbefore.xml"/>
                <CKLOG FILE=".\TC065\01\out\cksbefore.xml"/>
            </LOGBEFORE>
            <DELETEBLOCK BLKID="0x40" IGNORE_ERR="true" IGNORE_RELINK="true"/>
            <LOGAFTER>
                <IBLOG FILE=".\TC065\01\out\ibafter.xml"/>
                <CKLOG FILE=".\TC065\01\out\cksafter.xml"/>
            </LOGAFTER>
        </IBMOD>
    </HEXMOD>
</CONF>
```

As seen from the above picture, a command named DELETEBLOCK is given inside Infoblock.xml This command takes care of deleting the complete Infoblock. The details about the attribute is as follows

Elements	**Children or Attributes**	**Description**
DELETEBLOCK		This tag is under IBMOD tag. It is optional and can appear any number of times. This tag has to appear before VALIDATE tag in the xml file
	BLKID	This is a mandatory attribute. This attribute specifies the block id of the infoblock to be deleted.
	IGNORE_ERR	This is a mandatory attribute. This is a Boolean attribute, i.e. it can take only true or false as values If this attribute is true then even if the block id , which is given for deletion, is not present in the hex file it is ignored. If it is false HexmodX aborts with an error saying the the block is not present in the file
	IGNORE_RELINK	This is a mandatory attribute. This is a Boolean attribute, i.e. it can take only true or false as values If this attribute is true then after the deletion of the info block, all other blocks will NOT be re linked. If it is false HexmodX will re link all the remaining blocks in the same order as it was linked earlier

2.1.4.6 Detailed logging of Infoblock Elements

HexmodX tool provides an easy way for a user to write the addresses and the contents of all the basic elements of an Infoblock from a hex file to the xml file. Basic elements of Infoblock mean all the header information about the Infoblock, which will be present in the BASIS INFOBLOCK section of a block. Along with basis Infoblock, even the information about the checksum blocks and the epilogue contents and the respective addresses of the epilogue elements will also be print. The xml file which contains all these addresses and contents is generally termed as ADDRESS LAYOUT MODEL. To

achieve this, a command named DETAILEDLOG will have to be mentioned in the Infoblock.xml file. The details about the command and the attributes are listed below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="www.rbin.com/mds-1
M:\hexmod_paj3kor\hexmod17\schemas\infoblock.xsd">
    <!-- This is the input to the infoblock dll -->
    <HEXMOD>
        <IBMOD>
            <IBBUILD>
                <BLKLINK START="0x80820000" NXTBLK="0x809C0000"/>
                <BLKLINK START="0x80810000" NXTBLK="0x80820000"/>
                <BLKLINK START="0x809C0000" NXTBLK="0x0"/>
                <BLKLINK START="0x80808000" NXTBLK="0x80810000"/>
            </IBBUILD>
            <LOGBEFORE>
                <IBLOG FILE=".\TC065\01\out\ibbefore.xml"/>
                <CKLOG FILE=".\TC065\01\out\cksbefore.xml"/>
            </LOGBEFORE>
            <DETAILEDLOG FILE=".\TC065\01\out\InfoblockAddress2.xml" INFOBLOCK="ALL"/>
            <LOGAFTER>
                <IBLOG FILE=".\TC065\01\out\ibafter.xml"/>
                <CKLOG FILE=".\TC065\01\out\cksafter.xml"/>
            </LOGAFTER>
        </IBMOD>
    </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
DETAILEDLOG		This tag is under IBMOD tag. It is optional and can appear any number of times. This tag has to appear before VALIDATE tag in the xml file
	FILE	This is a mandatory attribute. This attribute specifies the path and the name of the address layout model xml file.
	INFOBLOCK	This is a mandatory attribute. This attribute can take the name of the infoblock or the start address of the infoblock or a keyword “ALL” which indicates to take all infoblocks into consideration present in the hex file for printing the address layout model. Currently this filed understands the following infoblocks, if given as names. They are SB(Startup block), CB(Customer block), TP(Tuning Protection), CTP(Customer Turning Protection), ASW0(Application software 0), ASW1(Application Software 1), DS0(Data Set 0), DS1(Data Set1), VDS(Variant Data Set)
	REPORTINVALID BLOCKS	This is an optional boolean attribute. The default value of this flag is **false** .This attribute specifies whether any UKNOWN blocks with invalid data are to be written to the detaillog or not. A value of true writes the UNKNOWN block details to the detaillog and the value of false avoids any UNKNOWN block details to be written to the detaillog. A sample configuration looks as below.

```
<HEXMOD>
  <IBMOD>
    <IBBUILD>
        <BLKLINK START="0x80018000" NXTBLK="0x80014000" />
        <BLKLINK START="0x80014000" NXTBLK="0x80000000" />
        <BLKLINK START="0x80000000" NXTBLK="0x80080000" />
        <BLKLINK START="0x80080000" NXTBLK="0x80004000" />
        <BLKLINK START="0x80004000" NXTBLK="0x00000000" />
    </IBBUILD>
  <DETAILEDLOG FILE="AddressModel.xml" INFOBLOCK="ALL" REPORTINVALIDBLOCKS="true"
/>
  </IBMOD>
<HEXMOD>
```

REPORTINVALIDBLOCKS ="**true**" specifies that UKNOWN blocks, if any, present in the Hex file should be printed.

2.1.4.7 VRTE Infoblock

VRTE Infoblock memory layout is different from MDG1 and MDG2. Memory layout of VRTE hex file is configured in xml file. HexmodX parses this xml file and patches the metadata into output hex file. Sample xml file is below,

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1
C:\Work\HexmodX_Workspace_13.0.0\HexmodX_trunk\src\main\resources\schemas\i
nfoblock.xsd">
      <HEXMOD>
            <IBMOD>
                  <VRTE METADATA_FILE="metadata.xml" />

            </IBMOD>
      </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
VRTE		It is optional. This element supports VRTE Infoblock functionality.
	METADATA_FILE	It indicates the xml file which is configured with metadata for memory blocks in hex file.

2.1.4.7 MULTILINK CHAIN Infoblock for MDG2

MULTILINK chain is to realize future concepts like Hypervisor, it might be necessary to be able to build several LinkChain elements from one block. Therefore, the Infotable mechanism will be used. This will be supported only from one block in complete LinkChain. This feature is in development stage.

Elements	Children or Attributes	Description
MULTILINK		It is optional. This element supports Multilinkchain Infoblock functionality.
	START	It indicates the start address of the multi linkchain. It is mandatory

2.1.5 Checksum Module

Checksum modules are used to calculate the checksum over the range specified. This is triggered by keyword **CHKSUM**. It also takes the initial and final values for Checksum calculations. Checksum calculation depends on the chosen algorithm. This can be provided by Checksum algorithm names. Possible types of algorithms are {CRC32, ADD32, CRCxx, IFXCRC, ADD16 and ADD8}

This checksum module is loaded by Infoblock module for the checksum over Infoblock and Checksum Infoblock. Default CRC32 checksum algorithm will be used for the Checksum over Infoblock.

For checksum over the checksum blocks, depending on the index in the hex file (checksum Infoblock structure) the algorithm, respective algorithm will be invoked. The index for the selection of the algorithm is as shown below.

If Checksum range is not specified then HEXMODX will calculate checksum for MIN address and MAX address of the HEX file.

Index	Algorithm
0x1	ADD32
0x2	CRC32
0x10	ADD16
0x8	ADD8

Checksum algorithm can also be used to calculate the checksum over the hex data ranges and write the checksum result to output hex file or pre-defined address. This is helpful for the user to cross check the checksum results in the hex file. For this purposes, the user can use checksum module.

Checksum.xml should written by the user, and this has to be loaded in the main XML file. The sample XML file for Checksum module usage is as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\checksum.xsd">
    <HEXMODX>
        <CHKSUM OUTPUTFILE="xyz.txt" OUTPUTADDRESS="0xA0000000" AUTOALIGN=”true”>
            <CRC32 INVCRC = “false” />
            <CKSRANGE FROM="0xA0008000" LEN="0x80" STARTVAL="0xFADACAFE"
ENDVAL="0xCAFEAFFE"/>
            <CKSRANGE FROM="0xA0008000" LEN="0x80" STARTVAL="0xFADACAFE"
ENDVAL="0xCAFEAFFE"/>
        </CHKSUM>
    </HEXMODX>
</CONF>
```

Elements	Children or Attributes	Description
CHKSUM		CHKSUM element is used to trigger the checksum calculation routine. This has optional attributes. If both the attributes are not provided, error will be thrown. If both options are chosen, HexmodX will perform both the operations.
	OUTPUTFILE	Checksum module writes the checksum result to output file specified by OUTPUTFILE attribute. This is an optional attribute.
	OUTPUTADDRESS	Checksum module writes the checksum result to address specified by OUTPUTADDRESS attribute. Checksum over all the ranges will be calculated and final checksum result will be written to the location specified by OUTPUTADDRESS attribute. This is an optional attribute.
	HEXFILE	Specify the input hex file for which checksum value is calculated. It is an optional attribute.
	FILETYPE	Specify the type of the input hex file.
	IGNORELINECKS	Optional boolean attribute to ignore line checksum. Default value is false.
	AUTOALIGN	This attribute is present in CKSRANGE element. Automatic alignment for address ranges while calculating checksum. By default, automatic alignment is set to “false”. If user needs a check for checksum ranges to be aligned for 2 or 4 bytes, then this option must be set to “true” For ADD8 algorithm Alignment check is not done. For ADD16, the alignment check will set for 2 bytes For other algorithms, alignment check will be done for 4 bytes.

2.1.5.1 Checksum Algorithms

Following table describes the checksum algorithms provided by the HexmodX tool.

Elements	Children or Attributes	Description
CRC32		CRC32 is algorithm provided by the HexmodX application. This algorithm is triggered by CRC32 tag under CHKSUM tag.

	INVCRC	BOSCH specific enhancement to CRC32 algorithm. If this attribute is set to “TRUE”, this algorithm will be called after CRC32 checksum calculation. If this attribute is set to “FALSE”, this algorithm will not be called.
CRCxx		This is similar to CRC32 algorithm, however, Polynomial and its Width is configured by the user in the XML file.
	POLY	Specifies the Polynomial used for Checksum Algorithm CRCxx. This is required only for CRCxx algorithm.
	WIDTH	Specifies the width of the polynomial to be used. It varies from 8 to 32. This is required only for CRCxx algorithm.
ADD32	**INVADD32**	Checksum algorithm that is used for 32 bit addition. This attribute is used to INVERT the calculated checksum result. This is an optional Boolean attribute. Default value “false” When the value is “true”, the inverted calculated result will be written to the address / file (chosen by **OUTPUTFILE** or **OUTPUTADDRESS**)
	COMPENSATION BYTES	Specifies whether compensation byte feature is to be enabled or not. This feature is enabled if the value of this attribute is “true”. By default the value of this attribute is set to “false” i.e. this feature has to be enabled explicitly. For detail information about **COMPENSATIONBYTES** is provided at the end of the table.
IFXCRC		Variant of CRC algorithm, BOSCH specific. Specifically used for ABM balancing. However, User can also use it by specifying IFXCRC tag in the Checksum module configuration file.
ADD16		Checksum algorithm that is used for 16 bit addition. ECU side implementation had to calculate the algorithm in a way, that the last 32 bits will be interpreted as only one 32 bit value. HexmodX balance the range from start to end address by adding **STARTVAL** (32 bit) and 16 bit values from start to end of the range, except last four bytes and subtract result from **ENDVAL** (expected value, which is 32bit). Balance value will be written to the last four bytes.
	INVADD16	This attribute is used to INVERT the calculated checksum result. This is an optional Boolean attribute. Default value “false” When the value is “true”, the inverted calculated result will be written to the address / file (chosen by **OUTPUTFILE** or **OUTPUTADDRESS**)
	COMPENSATION BYTES	Specifies whether compensation byte feature is to be enabled or not. This feature is enabled if the value of this attribute is “true”. By default the value of this attribute is set to “false” i.e. this feature has to be enabled explicitly. For detail information about **COMPENSATIONBYTES** is provided at the end of the table.

ADD8		Checksum algorithm that is used for 8 bit addition. In the ECU side implementation, Calculation of ADD8 algorithm has to be such a way that, last 32 bits will be interpreted as only one 32 bit value. HexmodX balance the range from start to end address by adding **STARTVAL** (32 bit) and 8 bit values from start to end of the range and subtract result from **ENDVAL** (expected value, which is 32bit). If **FROM** and **LEN** is not specified in the xml file then **MINADDRESS** is taken for **FROM** and **MAXADDRESS** is used to calculate **LEN.**
	INVADD8	This is the optional Boolean attribute which can be specified in xml file along with ADD8. If **INVADD8** is **True** then HEXMODX will invert the Final calculated checksum obtained from ADD8 algorithm and write the same to the Address / File specified in the attributes **OUTPUTADDRESS / OUTPUTFILE.** Default value "**false**"
MODULO256	**DISPLAYADDRESSLENGTH**	This is an optional attribute which specifies the format in which the addresses are to be written to the given output file. The output file is in text format. It takes values from 0x01 to 0x04. The detail description of this algorithm is as explained below.
CRC32CCITT		A new algorithm **CRC32CCITT** is introduced into HexmodX. The checksum is calculated from the lowest address to the highest address in the hex file. Thus the checksum is calculated for the whole hex file. The calculated checksum is written to two log files, an xml file and a txt file. **CKSRANGE** is not used for this algorithm.
	OUTPUTXMLFILE	It is a log file generated in xml format. It contains Start address, End address and Checksum value.
	LOGFILE	It is another log file which contains Start address, End address, Data Size and Checksum value but generated in text file format.
CRC64		**CRC64** is algorithm provided by the HexmodX application. This algorithm is triggered by **CRC64** tag under **CHKSUM** tag.
	INVCRC	If this attribute is set to "TRUE", this algorithm will be called after CRC64 checksum calculation. If this attribute is set to "FALSE", this algorithm will not be called.
CRC16CCITT		**CRC16CCITT** is algorithm provided by the HexmodX application. This algorithm is triggered by CRC16CCITT tag under **CHKSUM** tag. This tag is followed by **CKSRANGE**.
SHA256		**SHA256** algorithm calculates 32 bytes checksum value. Address range is provided by **CKSRANGE.** If this element is not provided, whole hex file is considered.
SHA512		**SHA512** algorithm calculates 32 bytes checksum value. Address range is provided by **CKSRANGE.** If this element is not provided, whole hex file is considered.

MODULO256: Modulo256 is a checksum algorithm which is similar to ADD8 algorithm only difference being that the carry generated each time, by adding two bytes, is added back to the 8 bit sum. Thus the checksum generated will be always 1 byte. This feature

is requested by SRL team for Volvo customer. A sample configuration is as explained below.

```
<CHKSUM OUTPUTFILE="Modulo256ChecksumOutput2.txt">
        <MODULO256 DISPLAYADDRESSLENGTH="0x3"/>
        <CKSRANGE FROM="0x80040000" LEN="0x161800"/>
</CHKSUM>
```

DISPLAYADDRESSLENGTH="0x3" specifies that the addresses written to the output file should be 3 bytes i.e. in this case the output file looks as below.
0x800400 , 0x801A17 , BB . Where BB is the calculated checksum.
0x800500 , 0x801A27 , CC Where CC is the calculated checksum

CRC16CCITT: A sample configuration for CRC16CCITT is as follows.

```
<CHKSUM OUTPUTFILE="CRC16CCITTChecksumOutput.txt">
        <CRC16CCITT/>
        <CKSRANGE FROM=" 0x80820098" LEN=" 0x11FEDC" STARTVAL="0xFADECAFE"
ENDVAL="0xCAFEAFFE"/>
</CHKSUM>
```

2.1.5.1.1 COMPENSATIONBYTES

This feature is used to calculate the checksum of a block and store the checksum in the hex file. The checksum is written to an address with value 00 00 FF FF. The address to which the checksum is written does always contains the value 00 00 FF FF. But whenever a new checksum value has to be written to this address, the checksum value of 00 00 FF FF is preserved by writing the first two bytes with the two bytes of the checksum. The value 00 00 is replaced by the checksum value and the value FF FF is replaced by the value obtained by subtracting the obtained checksum value from FF FF. For example, let's assume we want to check from 0000 to 0200 and we want to write the result in 0100. Then the checksum over this range would be different before and after writing the checksum into 0100.

Below is the description how the compensation bytes feature works:

Usually the target address 0100 contains the data 00 00 FF FF.
Then we calculate the checksum from 0000 to 2000 and enter the result in 0100 and 0101.
Then we subtract the result from FFFF and enter the result hereof into 0102 and 0103.

Example:
Before: 00 00 FF FF
Calculated checksum: 34 B1, subtracted checksum: CB 4E
After: 34 B1 CB 4E.

If we then build the checksum again over this range we get the same result as before.

2.1.5.2 Defining Checksum Range

Following table describes XML tags for defining the data range for checksum calculations.

Elements	Children or Attributes	Description

CKSRANGE		Child element under CHKSUM tag for specifying the checksum range. Checksum over the specified range is triggered by this keyword. This has the attributes like FROM and LEN to specify the checksum range start address and length. STARTVAL and ENDVAL attributes contains the initial and final values for checksum algorithm.
	STARTVAL	Specifies the initial value for the checksum algorithm
	ENDVAL	Specifies the end value for the checksum algorithm
	FROM	Specifies the start address of the checksum range This attribute is **optional**. If not provided, the lowest address of the file shall be considered.
	LEN	Specifies the length of the checksum range This attribute is **optiona**l. If not provided, the whole file range shall be considered.

2.1.6 ABM Module

ABM module is used to validate the Alternate Boot Mode (ABM) area in the Hex File. ABM can be located at 4 different locations. They are PRIMARY, SECONDARY, EX_PRIMARY, EX_SECONDARY.

Checksum for Boot mode or ABM is calculated using CRC32 algorithm and result is updated in ABM.

HexmodX checks if Boot Mode Header ID(BMHDID) is 0xB359 or not, in a specified location. For example, if ID is present in PRIMARY location 0x8001FFE0, ABM checksum will be calculated using CRC32 algorithm and updated after 16 bytes from ABM start address and immediately followed by updation of checksum complement. Then HexmodX does not check other ABM locations.

If ID is not present in PRIMARY location, HexmodX will check other ABM locations.

These locations can be configured through the XML keywords. See below for more details.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\abm.xsd">
    <HEXMODX>
        <ABMMOD DEVICE="DEV5" >
            <PRIMARY START="0x8001FFE0"/>
            <SECONDARY START="0x8003FFE0"/>
            <EX_PRIMARY START="0x8080FFE0"/>
            <EX_SECONDARY START="0x8088FFE0"/>
        </ABMMOD>
    </HEXMODX>
</CONF>
```

Elements	Children or Attributes	Description
ABMMOD		ABMMOD element triggers the ABM processing operation.

	DEVICE	Device attribute is optional. It can contain the values IFX_65NM, IFX_40NM, TC2XX, TC3XX.
	MDG1	If it is set as false then checksum for ABM header of MEDC17 (HexmodX17) is calculated. It is an optional attribute and by default it is true.
	PRIMARY	**START** attribute specifies the **PRIMARY** ABM location .
	SECONDARY	**START** attribute specifies the **SECONDARY** ABM location .
	EX_PRIMARY	**START** attribute specifies the **EX_PRIMARY** ABM location .
	EX_SECONDARY	**START** attribute specifies the **EX_SECONDARY** ABM location .
	START	Attribute holds the address location.

HexmodX also validates ABM block checksum start (ChkStart) and end (ChkEnd) addresses whether it is 4-byte aligned or not.
If ChkStart is not 4-byte aligned, HexmodX would abort and throw with an error **"Start Address for checksum range in ABM is not 4 byte aligned, hence aborting the operation."**

IFX_40NM:

For IFX 40NM devices, HexmodX validates 0xFA7CB359 for ABM part and 0xB359 for BMI parts. Then, it updates the checksum for ABM and BMI part respectively. Please find the sample configuration below,

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\abm.xsd">
    <HEXMOD>
        <ABMMOD DEVICE=”IFX_40NM”>
            <PRIMARY START="0x800F0000"/>
        </ABMMOD>
        <BMI>
            <PRIMARY START="0xAF400000"/>
            <SECONDARY START="0xAF400200"/>
        </BMI>
    <HEXMOD>
</CONF>
```

2.1.7 Error Module

Error module is basically used by all the modules for error logging. Error module is used for displaying messages to the users either on console window or in the log file. XML file is used for configuring the settings.

These settings can be part of XML files provided for each module as shown in Main Module

Each module can have its own error module setting. If not provided, settings of Main Module will be chosen.

Its not mandatory to have error module settings. By default following settings are chosen.

```
<ERRMOD>
```

```
    <LOG FILE="HexmodX.log"/>
    <MSGLOG LVL="1"/>
    <MSGCONSOLE LVL="1"/>
    <PROGABORT LVL="3"/>
</ERRMOD>
```

Elements	Children or Attributes	Description
ERRMOD		Child element of HEXMODX to trigger the error object creation for error handling.
	LOG	Child element of ERRMOD to specify the log file name using FILE attribute.
	FILE	Error log file name
	MSGLOG	User can choose this keyword to filter the messages to be written in LOG file. If user chooses attribute LVL value as 1, all messages including INFO are written to LOG file specified by LOG tag. default : message level is 1
	MSGCONSOLE	User can choose this keyword to filter the messages to be re-directed to console window. If user chooses attribute LVL value as 2, all messages above INFO (i.e., WARNING & ERROR) messages are displayed on the console. default : message level is 1
	PROGABORT	User can choose this keyword to abort the program for message level set. If user chooses attribute LVL value as 2, program aborts if any WARNING or ERROR messages occur.
	LVL	1 – INFO (information messages about the processes) 2 – WARNING (warning messages) 3 – ERROR (error messages) INFO messages generally used for displaying the information about what is happening while HexmodX execution. **Level is 1** WARNING messages provides users some hint about some actions happened which is not expected/chosen by user. **Level is 2** ERROR messages gives hint that some wrong actions happened. This provides the trigger to program abort. **Level is 3**

2.1.8 Compress Module

Compress module works based on the LZRB algorithm. Currently only LZRB algorithm is implemented.

HexmodX will throw an error if there are gaps present in input hex file. To avoid this error, fill the gaps using FILLPATTERN attribute then HexmodX will execute successfully.

The XML configuration of these options are as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1compress.xsd">
    <HEXMOD>
        <COMPRESS >
            <RANGE FROM="0x9000" TO="0x19000" AT=”0x00”
OUTPUTFILE=".\TC014\01\out\HexmodXtest2.hxf"/>
        </COMPRESS>
    <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
COMPRESS	**RANGE**	Child element of HEXMODX. It initiates the compress operation. Following tags appear under this. **RANGE**: This supplies range information for compression.
RANGE	**FROM** **TO** **OUTPUTFILE** **AT** **FILLPATTERN**	Range tag has the following attributes. **FROM**: Start address from where compression begins. If not specified, the default value will be considered as minimum address of the data model. **TO**: End address from where compression ends. If not specified, the default value will be considered as maximum address of the data model. **AT**: The address at which it needs to be written to the file. If not specified, the default value will be considered as 0x00. **OUTPUTFILE**: If the output file type is not specified, the default file type is taken as bin. **FILLPATTERN:** fill the gaps using fill pattern value.

2.1.9 Decompress Module

Decompress module works based on the LZRB algorithm. Currently only LZRB algorithm is implemented.

HexmodX will throw an error if there are gaps present in input hex file.

The XML configurations of these options are as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1compress.xsd">
  <HEXMOD>
      <DECOMPRESS >
          <RANGE FROM="0x9000" TO="0x19000" AT=”0x00” OUTPUTFILE=".\TC014\01\out\HexmodXtest2.hxf"/>
      </DECOMPRESS>
```

```
    <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
DECOMPRESS	**RANGE**	Child element of HEXMODX. It initiates the decompress operation. Following tags appear under this. **RANGE**: This supplies range information for decompression.
RANGE	**FROM** **TO** **OUTPUTFILE** **AT**	Range tag has the following attributes. **FROM**: Start address from where decompression begins. If not specified, the default value will be considered as minimum address of the data model. **TO**: End address from where decompression ends. If not specified, the default value will be considered as maximum address of the data model. **AT**: The address at which it needs to be written to the file. If not specified, the default value will be considered as 0x00. **OUTPUTFILE**: If the output file type is not specified, the default file type is taken as bin.

2.1.10 HexComp Module

HexComp module is used to compare the two ranges of data in hex files. HexComp also generates a report in cmp file which describe the information about the comparison.

The address and the file information can be configured through the XML configuration files. Please have a look at the sample xml file below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1.\schemas\HexCompare.xsd">
    <HEXMODX>
        <HEXCOMPARE LOG=”HexComp.cmp” >
            <COMPARE EXIT=”false” >
                <RANGE HEXFILE="file1.hex" START="0x80008000" LEN="0xFF"
TYPE="IHEX"/>
                <RANGE HEXFILE="file2.hex" START="0x90009000" LEN="0xFF"
TYPE="IHEX"/>
            </COMPARE>
        </HEXCOMPARE>
    <HEXMOD>
</CONF>
```

You can also compare the two different ranges from the same file. In such cases, you have to just specify the same file name in both RANGE attributes.

Elements	Children or Attributes	Description
HEXCOMPARE		HEXCOMPARE element triggers the HEXCOMPARE processing operation.

	LOG	**LOG** attribute specifies the **CMP** log name location. This file will contain the information of the files compared, location, and result of comparison.
COMPARE		COMPARE element triggers the actual processing operation.
		attribute specifies the whether HexmodX should or continue if one of the comparison fails. If this tag is **TRUE** then HexmodX will the HexComp module.
RANGE		RANGE tag has the information of the hex file name, address and length to be compared.
	HEXFILE	**HEXFILE** Attribute holds the file name from where the data to be fetched.
	START	**START** Attribute holds the address from where the data to be compared.
	LEN	**LEN** Attribute holds the length of the data to be compared.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.

2.1.10 Patch and Check Module

PatchAndCheck module is used to patch/check/view hex data in the hex file. Individual operations are explained as follows.

Patch operation is used for patching the hex file data with HEX or ASCII values at the specified address given in the configuration xml. Check operation is used for checking the value in the hex file with the value specified in the xml file for the specified address. View operation is used for viewing the hex content at the specified address given.

Sample xml file for PATCH operation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance xsi:schemaLocation="www.rbin.com/mds-1 \schemas\patchcheckdll.xsd">
    <HEXMOD>
        <PATCHCHECK RESULT-LOC="c:\temp\PatchInfo.xml">
            <ADDRESS NAME="ACCD_DebExcTempOk_C" VALUE="0xFFFFFFFF" ACTION="PATCH"
CUSTOMER="VW" TYPE="IHEX" FILENAME="sameplHex.hex" LEN="0">
                <LOCATION ADDRESS="0x1C297600"/>
            </ADDRESS>
        </PATCHCHECK>
    <HEXMOD>
</CONF>
```

This sample Patch operation patches the **0xFFFFFFFF** at **0x1C297600**.

Elements	**Children or Attributes**	**Description**
PATCHCHECK		PATCHCHECK element triggers the patch and check processing operation. This operation can be repeated for each patch/check/view operation.

	RESULT-LOC	**RESULT-LOC** attribute specifies the **XML** log name location. This file will contain the information of the files patched, name, and value, and location, type of patching and result of patching.
ADDRESS		ADDRESS element triggers the actual processing operation.
	NAME	**NAME** attribute specifies the name of the label / data to be patched. This is an optional element in the xml.
	VALUE	**VALUE** Attribute holds the value to be patched in the hex file.
	ACTION	**ACTION** Attribute the action to be performed, this can be PATCH/CHECK or VIEW. This is mandatory field. • **PATCH** action is used for patch the hex data with the given value at the given location. • **CHECK** action is used for check the hex data with the given value at the given location. • **VIEW** action is used for viewing the hex data with the given length at the given location.
	CUSTOMER	**CUSTOMER** Attribute holds the name of the customer. This is optional attribute.
	TYPE	This is a mandatory attribute. Specifies the type operation. HexmodX supports three formats. • Intel Hex format (IHEX) • ASCII format • REF type, if this is specified then **VALUE** tag user has to give an address. Patch and check will fetch the data from this address and use that as **VALUE.** Sample is given at the end of this table.
	FILENAME	**FILENAME** Attribute the name of the Hex file user is patching. This is mandatory field.
	LEN	**LEN** Attribute holds the length of the data o be retrieved from hex data; this is not used for PATCH / CHECK operation. This is used only in VIEW operation.
LOCATION		LOCATION tag is used for specifying the address of the hex file where the operation has to be carried out.
	ADDRESS	**ADDRESS** Attribute holds the address from where the data to be patched/checked/viewed.
FUNCTION		FUNCTION tag is used for finding out the address in the hex file based on the function NAME. This acts as accessing direct interface functions.
	NAME	This is a mandatory attribute if FUNCTION tag is used. Presently HexmodX supports two functions. • GetAddress • GetInfoTableAddress These features are explained section (GetAddress and GetInfoTableAddress).
PARAMETERS		PARAMETER has the information about the start address, offset and the level.
	NAME	NAME attribute holds the information about whether VALUE is STARTADDRESS, OFFSET1, OFFSET2 or LEVEL.
	VALUE	VALUE holds the actual hex address or the hex values of the offset. This attribute hold the values of type "**exhexBinary**".

PATCH Operation:
Sample for using REF type feature.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance xsi:schemaLocation="www.rbin.com/mds-1 \schemas\patchcheckdll.xsd">
    <HEXMOD>
        <PATCHCHECK RESULT-LOC="c:\temp\PatchInfo.xml">
            <ADDRESS NAME="ACCD_DebExcTempOk_C" VALUE="0x80008000" ACTION="PATCH"
CUSTOMER="VW" TYPE="REF" FILENAME="sameplHex.hex" LEN="4">
                <LOCATION ADDRESS="0x1C297600"/>
            </ADDRESS>
        </PATCHCHECK>
    <HEXMOD>
</CONF>
```

Here, pc module will fetch the data from 0x80008000 (address given in the VALUE tag) and use that data as the VALUE for the specified ACTION.

CHECK Operation:

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance xsi:schemaLocation="www.rbin.com/mds-1 \schemas\patchcheckdll.xsd">
    <HEXMOD>
        <PATCHCHECK RESULT-LOC="c:\temp\PatchInfo.xml">
            <ADDRESS NAME="ACCD_DebExcTempOk_C" VALUE="0x80008000" ACTION="CHECK"
CUSTOMER="VW" TYPE="REF" FILENAME="sameplHex.hex" LEN="4">
                <LOCATION ADDRESS="0x1C297600"/>
            </ADDRESS>
        </PATCHCHECK>
    <HEXMOD>
     </CONF>
```

Here, the data present at the source address 0x80008000 (value given in VALUE attribute) is compared with data present at destination address 0x1C297600. If they are same, check result is SUCCESS else FAILED.

2.1.10.1 GetAddress, GetInfoTableAddress and GetContentIDAddress

GetAddress tag is used to locate an address at an offset specified from a pointer address. Sample xml for GetAddress is as follows.

```
<ADDRESS NAME="ACCD_DebExcTempDef_C" VALUE="0xCAFEDEFE" ACTION="VIEW" CUSTOMER="VW"
TYPE="IHEX" FILENAME="sample.hex" LEN="4">
        <FUNCTION NAME="GetAddress">
            <PARAMETERS NAME="STARTADDRESS" VALUE="0x80800024"/>
            <PARAMETERS NAME="OFFSET1" VALUE="0x1000"/>
            <PARAMETERS NAME="LEVEL" VALUE="0x3"/>
        </FUNCTION>
</ADDRESS>
```

Since the level is specified as 3, HexmodX will iterate the 3 times to get the final address. Final address is calculated as follows.Consider the sample hex data.

```
:020000048080FA
:1000000000AAAAAAAAAAAAAAAAAAAA97
:10001000000000000000000000000000000000E0
:10002000000000008080006000000000000000D0
:10003000000000000000000000000000000000C0
:10004000000000000000000000000000000000B0
:10005000000000000000000000000000000000A0
:10006000808000940000000000000000000000090
:10008000000000000000000000000000000000070
:10009000000000008080008000000000000000060
:1000A000000000000000000000000000000000050
```

In first Iteration, pc module will fetch 4 bytes data at the given address, at 0x80800024, which is 0x80800060.In the second iteration pc module will fetch the data at 0x80800060, which is 0x80800094. In the third iteration, pc module will fetch the data at 0x80800094, which is 0x808000A0. Finally this data will be added to the offset given, i.e. 0x1000 and return the final address, which is 0x808010A0.

GetInfoTableAddress tag is used to locate an address at given offsets. Here we need to specify OFFSET'S to get the final address. The user can specify as many OFFSET'S as he wishes. In the below example we have take 2 OFFSET'S .The Sample.xml for GetInfoTableAddress is as follows.

```
<ADDRESS NAME="ACCD_DebExcTempDef_C" VALUE="0xCAFEDEFE" ACTION="VIEW"
                 CUSTOMER="VW" TYPE="IHEX" FILENAME="sample.hex" LEN="4">
        <FUNCTION NAME="GetInfoTableAddress">
            <PARAMETERS NAME="STARTADDRESS" VALUE="0x80800020"/>
            <PARAMETERS NAME="OFFSET1" VALUE="0x4"/>
            <PARAMETERS NAME="OFFSET2" VALUE="0x12"/>
        </FUNCTION>
</ADDRESS>
```

Consider the same sample above hex data.

In the first iteration, pc module adds the start address with the OFFSET1, i.e. 0x80800020 + 0x4, which is 0x80800024, and fetches the data at that location, i.e. 0x80800060. In the second iteration, new address will be added to the OFFSET2, 0x80800060+0x12, which is 0x80800072, and return as the final address.

GetContentIDAddress tag is used to locate the content ID address. Here we need to specify the start address of the block and the content ID in that block to get the final address. The start address of the block must be valid and must contain the content ID in the infotable. The Sample.xml for GetContentIDAddress is as follows.

```
<ADDRESS NAME=" BLOCKBLUSTER " VALUE="0xCAFEDEFE" ACTION=" PATCH "
                 CUSTOMER="String" TYPE="IHEX" FILENAME="sample.hex" LEN="4">
        <FUNCTION NAME="GetContentIDAddress">
            <PARAMETERS NAME="STARTADDRESS" VALUE="0x80000020"/>
            <PARAMETERS NAME="CONTENTID" VALUE="0x0080"/>
        </FUNCTION>
</ADDRESS>
```

The contentID address is present in the last 4 bytes of the infotable and is identified by the content ID found as first two bytes.

2.1.10.2 Patch and Check log file (PatchInfo.xml)

Patch check module generates xml file as the output information file, which has the information about the NAME, ACTION, NEW_VAL, OLD_VALE, TYPE etc. Sample PatchInfo file for a VIEW operation is as follows.

```
<LOG>
    <PATCHCHECK>
        <LABELS TYPE="ASCII">
            <NAME> ACCD_DebExcTempDef_C</NAME>
            <FILE>sample.hex</FILE>
            <ADDRESS>0x80800068 </ADDRESS>
            <OLD_VAL>0x80800094</OLD_VAL>
            <NEW_VAL> </NEW_VAL>
            <RESULT>SUCCESS</RESULT>
            <INFO>VIEW</INFO>
            <PHY-VAL/>
        </LABELS>
    </PATCHCHECK>
</LOG>
```

For the VIEW action, only OLD_VAL will be updated, and for the PATCH and CHECK operation, both NEW_VAL as well as the OLD_VAL will be updated. In NEW_VAL, VALUE tag in the input file will be written and in the OLD_VAL, actual value in the hex file will be written.

2.1.11 CleanUp Module

Cleanup module is created to erase the contents of TPROT (Tuning Protection) and CTPROT (Customer Tuning Protection Block) out from Hex files. The main reason for doing thing is to protect since both sections which are not to be read out.

After removing the TPROT and CTPROT from the hex file, the remaining hex file must be signable, must be good to generate signatures. Therefore it is not sufficient to remove the TPROT and CTPROT totally because in that case the remaining hex file can't be trimmed (checksums) and signatured.

Using the cleanup module, it is possible to do three kind of operation, Cleaning the TPROT and CTPROT section, Merging the TPROT and CTPROT back to the hex file and Checking the Content of the hex file to verify cleaned or not

2.1.11.1 Cleaning the TPROT and CTPROT

The contents of tuning protection (TPROT) and customer tuning protection (CTPROT) sections are confidential material. If Hex files are used outside Robert Bosch, the contents of TPROT and CTPROT have to be removed using this HexmodX extension.

Sample xml file for CleanUp operation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance xsi:schemaLocation="www.rbin.com/mds-1 \schemas\cleanup.xsd">
    <HEXMOD>
```

```
        <CLEANUP>
                < IBLK START="0x80018000"/>
        </ CLEANUP >
    </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
IBLK		IBLK element triggers the cleanup processing operation.
	START	START contains the address of the first infoblock in the chain.

2.1.11.2 Cleaned Blocks Serialization

While cleaning the hex file, only cleaned blocks can be serialized into separate hex file along with cleaned hex file. I.e. it contains all the blocks along with cleaned blocks.

Sample xml file for CleanUp operation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance xsi:schemaLocation="www.rbin.com/mds-1 \schemas\cleanup.xsd">
    <HEXMOD>
        <CLEANUP NAME=”cleanedBlocks.hex” TYPE=”IHEX”>
                < IBLK START="0x80018000" />
                < IBLK START="0x80014000" />
        </ CLEANUP >
    <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
CLEANUP		Cleans OTP blocks present in hex file.
	NAME	NAME specifies the name of the hex file for cleaned blocks. This attribute is optional.
	TYPE	TYPE specifies the type of hex file. Default type is IHEX. This attribute is optional.
IBLK		IBLK element triggers the cleanup processing operation. It can be present one or more times inside CLEANUP element.

2.1.12 Encryption Module

Encryption module encrypts the block of data using AES encryption mechanism.

Sample xml file for Encryption operation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 .\schemas/encrpyt.xsd">
    <HEXMOD>
        <ENCRYPT ALGO="AES/CBC/PKCS5Padding" KEY="0xffffffffffffffffffffffffffffffff">
            <BLOCK START="0x80030000" LEN="0x500"/>
        </ENCRYPT>
        <ENCRYPT ALGO="AES/CBC/PKCS5Padding">
            <BLOCK START="0xD8000000" LEN="0x400"/>
        </ENCRYPT>
```

```
        <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
ENCRYPT		ENCRYPT element triggers to encrypt the data.
	ALGO	ALGO specifies the name of the algorithm. This attribute is mandatory.
	KEY	KEY specifies the secret key which is used for converting plain text into cipher text. This attribute is optional.
	VECTOR	VECTOR is used to initialize the cipher to encrypt the data. This attribute is optional.
BLOCK		BLOCK element to specify the block of data.
	START	Start address of the block.
	LEN	Len of the data to encrypt.

2.1.14 Groovy Command Line Parameters.

Please finds the steps below, explaining how arguments can be passed to bean shell script:

1. While invoking HexmodX using BSH, arguments can be passed as shown below:

   call C:\temp\HexmodX\bin\HexmodX.cmd /HexmodXBShell.bsh –args "./../../HEXMODX/_log/HexmodX.log;medc18_tmp.hex;0x8002000;./../../HEXMODX/_out/medc18_out_HexmodX.hex"

   Please be noted that the parameter list is included within quotes(“”). The argument list is indexed from 1. You can pass n number of arguments to the script. Each argument is separated by a semicolon(“;”).

2. HexmodX adds these arguments to a list starting from index 1. The arguments can be accessed from BSh file.

3. Below is shown how to access the arguments in the script.

   A method call getProperty(“index of argument”) can be used to fetch argument value as below:

   HexmodXLogger.setLogFilePath(getProperty("1"));//getProperty(“1”) returns log file path which is first argument

   hexFileParser.parse(hexObject,getProperty("2"),true);//getProperty(“2”) returns hex file path which is second argument.

   And so on..

2.1.15 ConfigHexmodX module

ConfigHexmodX module generates xml (configuration) files for HexmodX to perform operations during MDG Build. Invocation or option (.opt) file is given as input for HexmodX to generate xml files. HexmodX passes it to ConfigHexmodX module. It will process opt file and generates xml files.

To invoke in command line, open command prompt and execute with following command,

HexmodX.cmd –optionsfile options_cfg_HexmodX_merge.opt

HexmodX generates xml files depending on switches and its values specified in opt file.

2.1.15.1 Trimming or extraction of infoblocks in hex file using ConfigHexmodX module.

To extract the trimmed infoblocks in hex file, ConfigHexmodX requires some switch options in opt file.
The following options are mandatory for this extraction of trimmed infoblocks,

--trimmed_hex_file_infoblocks_hex _bin/swb/filegroup/results/prj_name_trimmed_infoblocks.hex
--trimmed_hex_file_infoblocks asw0,ds0,vds
--trimmed_hex_file_infoblocks_out _gen/swb/module/HexmodX/trimmed_infoblocks_output.xml

trimmed_hex_file_infoblocks_hex options specifies the output hex file name where extracted infoblocks are serialized.
trimmed_hex_file_infoblocks option specifies the name of the infoblocks to be extracted from hex file.
trimmed_hex_file_infoblocks_out specifies the xml file using which HexmodX extract the infoblocks using ranges specified within xml file.

2.1.15.2 Generation of additional bin file

To generate additional bin file, following option is required in opt file. It is applicable only for Merge Clean Checksum functionality of ConfigHexmodX module. For example,

--clean_checksum_bin _gen/swb/module/HexmodX/MMD104VMAC1483_N6A0_CORETASK0_clean.bin

2.1.16 ITRAMS Module

ITRAMS module calculates signature for data part of the ITRAMS hex files and calculates checksum using CRC32 algorithm for header part of that hex files.

Sample XML file for the ITRAMS operation as follows,

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\itrams.xsd">
    <HEXMOD>
        <ITRAMS>
            <SIGNATURE MEMLAY_FILE="iTRAMS_Memlayout.xml"
PRIVATE_KEY="sigsso_private.key"/>
        </ITRAMS>
 <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
SIGNATURE		SIGNATURE element triggers to generate signature for the given data.
	MEMLAY_FILE	MEMLAY_FILE Memlay xml file contains the address details about the hex file.
	PRIVATE_KEY	PRIVATE_KEY specifies the private key file.

2.1.16 Decryption Module

Decryption module decrypts the block of data using AES decryption mechanism.

Sample xml file for decryption operation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--Sample XML file generated by XMLSPY v5 rel. 3 U (http://www.xmlspy.com)-->
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1 .\schemas/encrpyt.xsd">
    <HEXMOD>
            <DECRYPT ALGO="AES/CBC/NoPadding"
KEY="0xffffffffffffffffffffffffffffffff">
                <BLOCK START="0x80030000" LEN="0x500"/>
            </DECRYPT>
    <HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
DECRYPT		DECRYPT element triggers to encrypt the data.
	ALGO	ALGO specifies the name of the algorithm. This attribute is mandatory. **Currently only AES/CBC/NoPadding is supported.**
	KEY	KEY specifies the secret key which is used for converting plain text into cipher text. This attribute is optional.
	VECTOR	VECTOR is used to initialize the cipher to decrypt the data. This attribute is optional.
BLOCK		BLOCK element to specify the block of data.
	START	Start address of the block.
	LEN	Length of the data to decrypt.

2.1.17 VRTE Security Module

This module provides security features such as MAC value generation, keys and data encryption, hash, signature and certificate generation.

It consists of the below features,

- MAC value generation
- Keys and Data encryption

- Hash generation
- Signature and Certificate generation
- Signed hex file generation

2.1.17.1 MAC functionality

MAC functionality generates MAC value for VRTE memory blocks in hex file.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 ./schemas/security.xsd">
      <HEXMOD>
            <SECURITY>
                  <MAC VRTE_CONF_FILE="metadata.xml" KEYS_FILE="keys.xml" >
                        <INPUT_FILE FILENAME="input.hex" TYPE="IHEX" />
                        <OUTPUT_FILE FILENAME="output.hex" TYPE="IHEX" />
                  </MAC>
            </SECURITY>
      </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
MAC		**MAC** element triggers to generate MAC value.
	VRTE_CONF_FILE	VRTE_CONF_FILE It has xml file which contains metadata for memory blocks in hex file.
	KEYS_FILE	KEYS_FILE specifies the key file.
	ALGO_NAME	It indicates the algorithm name. This attribute is optional. Default value is **AESCMAC**.
	TYPE	This attribute is optional. To generate secure call back mac value, specify the value as **SECURECALLBACK**. To generate Boot manager block mac value, specify the value as **BOOTMANAGERBLOCK**
	INPUT_FILE	This element specifies the input file.
	OUTPUT_FILE	This element specifies the output file.
INPUT_FILE	FILENAME	Specifies the input file name.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
OUTPUT_FILE	FILENAME	Specifies the output file name.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	LYT	Specifies the layout of the hex file. i.e. length of each record / line.

2.1.17.2 Encryption functionality

Encryption functionality encrypts the keys configured in metadata xml file and keys xml file.
This functionality also encrypts the data in the hex file if DATA_ENCRYPTION attribute is true.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1 c:/schemas/security.xsd">
      <HEXMOD>
            <SECURITY>
                  <ENCRYPTION VRTE_CONF_FILE="metadata.xml" KEYS_FILE="keys.xml">
                        <INPUT_FILE FILENAME="input.hex" TYPE="IHEX" />
                        <OUTPUT_FILE FILENAME="output.hex" TYPE="IHEX" />
                  </ENCRYPTION>
            </SECURITY>
      </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
ENCRYPTION		**ENCRYPTION** element triggers to generate encrypted keys and data value.
	VRTE_CONF_FILE	VRTE_CONF_FILE It has xml file which contains metadata for memory blocks in hex file.
	KEYS_FILE	KEYS_FILE specifies the key file.
	ALGO_NAME	It indicates the algorithm name. This attribute is optional. Default value is **AES/CTR/NoPadding**.
	DATA_ENCRYPTION	It is boolean attribute. To enable data encryption, set the value to true. This attribute is optional. Default value is **false**.
	VALIDPATTERN_XML	This element generated valid pattern xml file. This valid pattern xml file contains valid pattern 1, 2 and 3 of all the blocks which are present in meta data xml file.
	INPUT_FILE	This element specifies the input file.
	OUTPUT_FILE	This element specifies the output file.
INPUT_FILE	FILENAME	Specifies the input file name.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
OUTPUT_FILE	FILENAME	Specifies the output file name.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	LYT	Specifies the layout of the hex file. i.e. length of each record / line.

2.1.17.3 Hash functionality

Hash generation functionality generates hash text file for memory blocks specified in metadata xml file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1 D:\HexmodX\schemas\mdg1tprot.xsd">
<HEXMOD>
      <HASH VRTE_CONF_FILE="metadata.xml">
            <INPUT INPUT="input.hex" OUTPUT="hash.txt" IGNORELINECKS="true"
ID="tag1" TYPE="IHEX" MODE="BigEndian" BLOCKAREA="header"></INPUT>
      </HASH>
</HEXMOD>
</CONF>
```

HASH		**HASH** element triggers to generate hash text file.
	VRTE_CONF_FILE	**VRTE_CONF_FILE** It has xml file which contains metadata for memory blocks in hex file.
	INPUT	This element specifies the input file.
INPUT	**INPUT**	This attribute specifies the input hex file name
	OUTPUT	It specifies the output hash text file.
	BLOCKAREA	This attribute indicates HEADER or PAYLOAD area. Value can be HEADER or PAYLOAD. Default value is HEADER.
	LYT	Specifies the layout of the hex file. i.e. length of each record / line.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
	MODE	It indicates the endianness of hex file. Value can be LittleEndian and BigEndian. This attribute is optional. Default value is LittleEndian.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	ID	Unique tag ID value to be written into output hash text file.

2.1.17.4 Signature and Certificate functionality

This functionality generates text file with Signature and Certificate for the input hash text file.

```
<?xml version="1.0" encoding="utf-8" standalone="no"?>
<CONF xmlns="www.rbin.com/mds-1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1  /security.xsd">
<HEXMOD>
      <SECURITY>
            <SIGNATURE MODE="ONLINE" SIGNATURE_ALGO="ECDSA">
                  <INPUT_HASH INPUT="hash.txt" OUTPUT="sign.txt" />
            </SIGNATURE>
      </SECURITY>
```

```
</HEXMOD>
</CONF>
```

SIGNATURE		**SIGNATURE** element triggers to generate sign text file with Signature and Certificate values.
	MODE	This attribute value can be ONLINE and OFFLINE. ONLINE indicates that connection to server is established to generate Signature and Certificate.
	SIGNATURE_ALGO	Default algorithm is RSA. Other supported algorithm is ECDSA.
	INPUT_HASH	Sign text file generation for the specified hash text file.
INPUT_HASH	**INPUT**	Specifies the input hash text file name.
	OUTPUT	Specifies the Sign text file name.
	KEYPATH	Specifies the KEYPATH value.
	OID	Specifies the OID value.
	AUTHBITS	Specifies the AUTHBITS value.

2.1.17.5 Signed hex file functionality

This functionality patches the Signature and Certificate from input sign text file to generate Signed output hex file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  /security.xsd">
<HEXMOD>
      <SECURITY>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_SIGN INPUTFILE="input.hex" INPUT_TXT="sign.txt"
IGNORELINECKS="true" TYPE="IHEX" MODE="LittleEndian" ID="tag1"
VRTE_CONF_FILE="metadata.xml" BLOCKAREA="payload">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX" />
                  </INPUT_SIGN>
            </SIGNATURE>
      </SECURITY>
</HEXMOD>
</CONF>
```

SIGNATURE		**SIGNATURE** element triggers to generate sign text file with Signature and Certificate values.
	MODE	This attribute value can be ONLINE and OFFLINE. ONLINE indicates that connection to server is established to generate Signature and Certificate.
	SIGNATURE_ALGO	Default algorithm is RSA. Other supported algorithm is ECDSA.
	INPUT_SIGN	Signed hex file generation for the specified sign text file.
INPUT_SIGN	**INPUTFILE**	This attribute specifies the input hex file name
	INPUT_TXT	Specifies the input sign text file name.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported
	MODE	It indicates the endianness of hex file. Value can be LittleEndian and BigEndian. This attribute is optional. Default value is LittleEndian.
	VRTE_CONF_FILE	It has xml file which contains metadata for memory blocks in hex file.
	BLOCKAREA	This attribute indicates HEADER or PAYLOAD area. Value can be HEADER or PAYLOAD. Default value is HEADER.

2.1.17.6 emmc HEX file signing

This provides security features hash, signature and certificate generation for emmc HEX files.

It consists of the below features,

- Hash generation
- Signature and Certificate generation
- Sign patching
- Sign calculation

2.1.17.6.1 Hash Generation

Hash generation functionality generates hash text file for specified **blockarea**.

Ranges for hash generation are considered as:

a. Header range : **0x20 - 0x2E0**

b. Payload range :

- **img = 0x00 - MaxAdd**
- **BIN = playloadOffset - SizeOfPayload**

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1 D:\HexmodX\schemas\mdg1tprot.xsd">
<HEXMOD>
      <HASH">
```

```
            <INPUT INPUT="input.hex" OUTPUT="hash.txt" IGNORELINECKS="true"
ID="tag1" TYPE="IHEX" MODE="BigEndian" BLOCKAREA="header" IMG="TRUE" >
                  <RANGES OUTPUT="hash_range.txt">
                        <RANGE FROM="0x80020000" TO="0x80020100"/>
                        <RANGE FROM="0x80030000" TO="0x80030100"/>
                  </RANGES>
            </INPUT>
      </HASH>
</HEXMOD>
</CONF>
```

HASH		**HASH** element triggers to generate hash text file.
	INPUT	This element specifies the input file.
INPUT	**RANGES**	Ranges specify ranges for hash calculation
	INPUT	This attribute specifies the input hex file name
	OUTPUT	It specifies the output hash text file.
	BLOCKAREA	This attribute indicates HEADER or PAYLOAD area. Value can be HEADER or PAYLOAD. Default value is HEADER.
	LYT	Specifies the layout of the hex file. i.e. length of each record / line.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
	MODE	It indicates the endianness of hex file. Value can be LittleEndian and BigEndian. This attribute is optional. Default value is LittleEndian.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported.
	ID	Unique tag ID value to be written into output hash text file.
	IMG	It is Boolean attribute. Used to specify **img** file is provided as input or not. Default value is false.
RANGES	**RANGE, OUTPUT**	**RANGES** tag provides provision to calculate HASH over a range of data.
	OUTPUT	**OUTPUT** attribute specifies the path/name of the input hex file. This is an optional attribute. This attribute provides file name for writing output file. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pick up the files from application directory. If **OUTPUT** is not specified in **RANGES**, the hash calculated under **RANGES** will go into **OUTPUT** file of the parent **INPUT** tag.
RANGE	**FROM, TO**	**RANGE** tag is used to specify the extremes of the range via FROM and TO attributes.
	FROM	**FROM** attribute is used to mark the start address of the range of hash calculation.
	TO	**TO** attribute is used to mark the end address of the range of hash calculatin.

2.1.17.6.2 Signature and Certificate generation

This functionality generates text file with Signature and Certificate for the input hash text file.

```
<?xml version="1.0" encoding="utf-8" standalone="no"?>
<CONF xmlns="www.rbin.com/mds-1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1  /security.xsd">
<HEXMOD>
      <SECURITY>
            <SIGNATURE MODE="ONLINE" SIGNATURE_ALGO="ECDSA">
                  <INPUT_HASH INPUT="hash.txt" OUTPUT="sign.txt" />
            </SIGNATURE>
      </SECURITY>
</HEXMOD>
</CONF>
```

SIGNATURE		**SIGNATURE** element triggers to generate sign text file with Signature and Certificate values.
	MODE	This attribute value can be ONLINE and OFFLINE. ONLINE indicates that connection to server is established to generate Signature and Certificate.
	SIGNATURE_ALGO	Default algorithm is RSA. Other supported algorithm is ECDSA.
	INPUT_HASH	Sign text file generation for the specified hash text file.
INPUT_HASH	**INPUT**	Specifies the input hash text file name.
	OUTPUT	Specifies the Sign text file name.
	KEYPATH	Specifies the KEYPATH value.
	OID	Specifies the OID value.
	AUTHBITS	Specifies the AUTHBITS value.

2.1.17.6.3 Sign patching

This functionality patches the Signature and certificate from input sign text file to generate Signed output hex file.
Signature is patched at :
Header - **0x700**
Payload - **0x100**
Certificate is patched at : **0x300**

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  /security.xsd">
<HEXMOD>
      <SECURITY>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_SIGN INPUTFILE="input.hex" INPUT_TXT="sign.txt"
IGNORELINECKS="true" TYPE="IHEX" MODE="LittleEndian" ID="tag1"
SIGN_ADDRESS="0x80080000" BLOCKAREA="payload">
```

```
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX" />
                  </INPUT_SIGN>
            </SIGNATURE>
      </SECURITY>
</HEXMOD>
</CONF>
```

SIGNATURE		**SIGNATURE** element triggers to generate sign text file with Signature and Certificate values.
	MODE	This attribute value can be ONLINE and OFFLINE. ONLINE indicates that connection to server is established to generate Signature and Certificate.
	SIGNATURE_ALGO	Default algorithm is RSA. Other supported algorithm is ECDSA.
	INPUT_SIGN	Signed hex file generation for the specified sign text file.
INPUT_SIGN	**INPUTFILE**	This attribute specifies the input hex file name
	INPUT_TXT	Specifies the input sign text file name.
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported
	MODE	It indicates the endianness of hex file. Value can be LittleEndian and BigEndian. This attribute is optional. Default value is LittleEndian.
	BLOCKAREA	This attribute indicates HEADER or PAYLOAD area. Value can be HEADER or PAYLOAD. Default value is HEADER.
	SIGN_ADDRESS	This attribute used to provide address to patch signature at that address.

2.1.17.6.4 Sign calculation

This feature is used to generate the signed hex file from the input hex file. Please find below the sample xml file to generate the sign text file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  /security.xsd">
<HEXMOD>
      <SECURITY>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_HEX INPUTFILE="input.hex" IGNORELINECKS="true"
TYPE="IHEX" MODE="LittleEndian" BLOCKAREA="payload">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX" />
                  </INPUT_HEX>
            </SIGNATURE>
      </SECURITY>
</HEXMOD>
</CONF>
```

SIGNATURE		**SIGNATURE** element triggers to generate sign text file with Signature and Certificate values.
	MODE	This attribute value can be ONLINE and OFFLINE. ONLINE indicates that connection to server is established to generate Signature and Certificate.
	SIGNATURE_ALGO	Default algorithm is RSA. Other supported algorithm is ECDSA.
	INPUT_HEX	Signed hex file generation for the specified sign text file.
INPUT_HEX	**INPUTFILE**	This attribute specifies the input hex file name
	IGNORELINECKS	It is boolean attribute. It ignores the line checksum issue if the value is true. This attribute is optional. Default value is false.
	TYPE	Specifies the input file type. Only IHEX, S19 and BIN are supported
	MODE	It indicates the endianness of hex file. Value can be LittleEndian and BigEndian. This attribute is optional. Default value is LittleEndian.
	BLOCKAREA	This attribute indicates HEADER or PAYLOAD area. Value can be HEADER or PAYLOAD. Default value is HEADER.

2.1.18 Cluster Linker HEX file patching

Cluster linker HEX file patching module will patch the symbol address and value in the given input HEX file. Cluster cml will be generated in build environment which has the input file details, symbol address and value details and output HEX file details

Sample XML file for patching symbol address in cluster HEX files.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="www.rbin.com/mds-1
file:///C:/Program%20Files/Bosch/Hexmod/schemas/baseio.xsd">
<HEXMOD>
<CLUSTERS>

<CLUSTER>

<FILEIN TYPE="IHEX" NAME="Core.hex" IGNORELINECKS="true" MODE="LittleEndian"
OVERWRITE="true">
<SYMBOL ADDRESS="0x80800000"  VALUE="0x5060709F" OVERWRITE="true"/>
<SYMBOL ADDRESS="0x80800010"  VALUE="0x67890123" OVERWRITE="true"/>
<SYMBOL ADDRESS="0x80800020"  VALUE="0x27890123" OVERWRITE="true"/>
<SYMBOL ADDRESS="0x80800030"  VALUE="0x80000000" OVERWRITE="true"/>
</FILEIN>
<FILEOUT TYPE="IHEX" NAME="./../../HEXMOD/_out/Cluster1_out.hex" LAYOUT="0x10"/

</CLUSTER>

</CLUSTERS>
</HEXMOD>
</CONF>
```

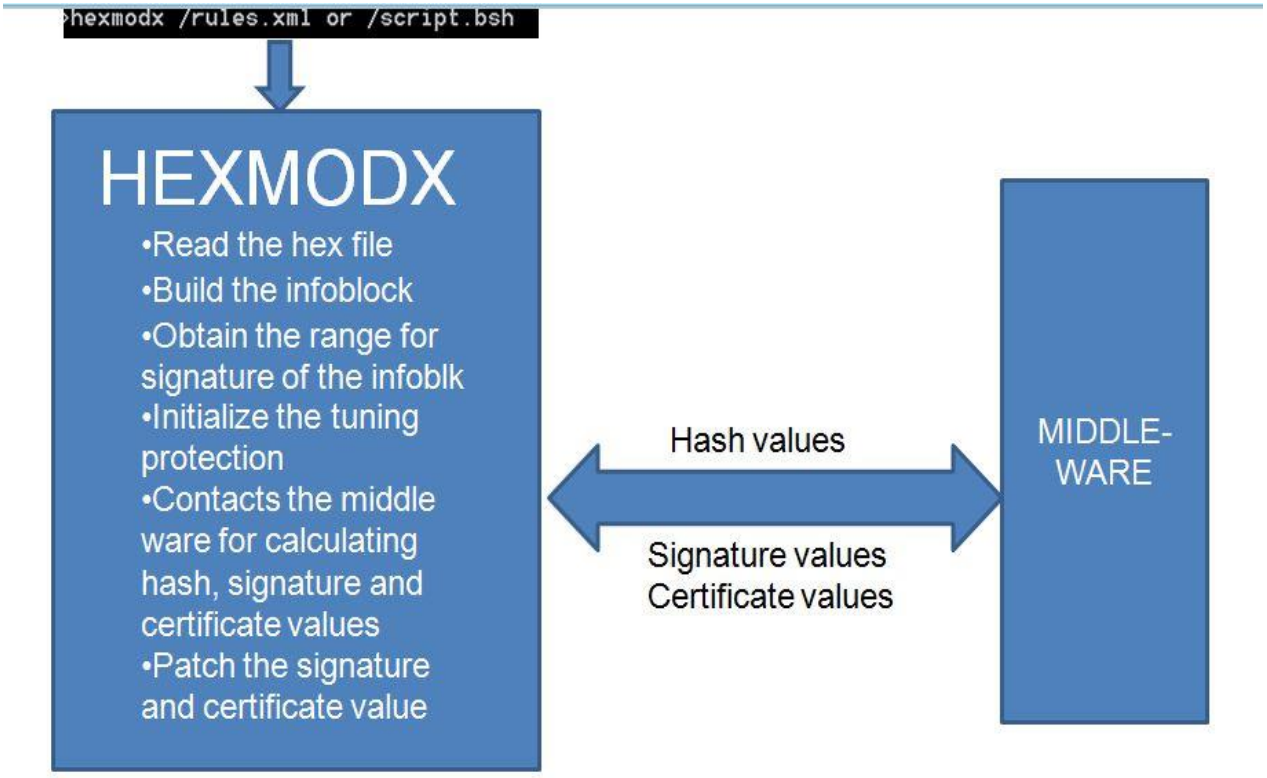
Elements	Children or Attributes	Description
CLUSTERS		
CLUSTER		
FILEIN	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.
	NAME	This is a mandatory attribute This attribute provides file name for input reading. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.
	OVERWRITE	This is an optional boolean attribute with default value “false” Overwrite option is used while merging multiple files read during INPUT operation. If this option is set to “true”, and if any duplicate address record found with existing data and data while reading subsequent files.
	MODE	This attribute defines the endianess of hex file. Either LittleEndian or BigEndian can only be specified for endianess. It is a mandatory attribute.
	SYMBOL	Symbol is an element which is used t to get address and value details from user.

SYMBOL	**ADDRESS**	This specifies the address details in the HEX file where value needs to be patched.
	VALUE	Value which need to be patched in the symbol address need to be given here.
	OVERWRITE	Overwrite is a Boolean attribute which need to be provided to overwrite th input values.
FILEOUT		FILEOUT tag is used to write the modified hex file contents into a file specified. This tag is a child of OUTPUT tag.
	TYPE	This is a mandatory attribute. Specifies the file type. HexmodX supports two file formats. • Intel Hex format (IHEX) • Motorola S-Record (S19) Using this attribute, user can specify the file type provided for input.
	NAME	This is a mandatory attribute This attribute provides file name for writing output file. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pickup the files from application directory.
	LYT	This attribute is used to configure the layout (byte length) of each data records, while writing the output file. This can be configured from “0x00” to “0xFF”.

3 MDG1 Tuning Protection

3.1 General

To generate signed hex file, HexmodX has dependency on Middleware library supplied by vendor ESCRYPT. A middleware component is available as a DLL that performs the hash, signature and certificate calculation for input data bytes. MDG1TPROT module of HexmodX is a wrapper around the middleware component. It provides interfaces to perform different operations on the middleware such as calculating and updating the signature and certificate for the memory blocks (Infoblocks) in hex file. The scenario is as depicted below,



3.2 Installation

HexmodX supports both 32-bit and 64-bit middleware. In local system, either 32-bit or 64-bit can only be installed. Note that only 64-bit Middleware is supported since 10.2.0 version.
To install Middleware, please contact ESCRYPT hotline **"ESCRYPT support (ETAS-PSC/EPE) <support@escrypt.com>".**

3.2.3 MDG1TPROT module

MDG1TPROT module is used for all tuning protection related operations on hex files. This module is used for hash calculation, signature generation and certificate generation.

MDG1TPROT module XML file is validated against the schema “mdg1tprot.xsd”, a schema file available within HexmodX. In main configuration file (e.g. rules.xml), this module can be named as “MDG1TPROT”.

3.2.3.1 Hash Calculation

This feature is used to calculate the hash value of a hex file. Please find below the sample xml to calculate the HASH values for INFOBLOCKS (memory blocks) and RANGES (user defined ranges).

```
<? Xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <HASH>
            <INPUT INPUT="medc18.hex" OUTPUT="hash.txt"
IGNORELINECKS="true" ID="tag1" TYPE="IHEX" MODE="BigEndian">
                  <INFOBLOCK INFOBLOCKADDR="0x80020000"/>
                  <INFOBLOCK INFOBLOCKADDR="0x80020000"/>
                  <RANGES OUTPUT="hash.txt">
                        <RANGE FROM="0x80020000" TO="0x80020100"/>
                        <RANGE FROM="0x80030000" TO="0x80030100"/>
                  </RANGES>
            </INPUT>
      </HASH>
</HEXMOD>
</CONF>
```

To generate hash values for HSM configuration area, specify HSM xml configuration file in HSM_FILE attribute of HASH element.

Elements	Children or Attributes	Description
HASH	**INPUT**	**HASH** tag specifies the hash operations. It can have multiple input tags.
	HSM_FILE	**HSM_FILE** attribute specifies HSM xml configuration in which start address and length of the data range for hash calculation of HSM configuration area is configured It is an optional attribute.
INPUT	**INPUT, OUTPUT, IGNORELINECKS, ID, TYPE AND MODE**	**INPUT** tag provides options to perform HASH calculation over an infoblock chain, a single infoblock or an over a range of data.
	INPUT	**INPUT** attribute specifies the path/name of the input hex file. This attribute provides file name for input reading. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pick up the files from application directory.

	OUTPUT	**OUTPUT** attribute specifies the path/name of the input hex file. This is a mandatory attribute. This attribute provides file name for writing output file. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pick up the files from application directory.
	IGNORELINECKS	This is an optional boolean attribute with default value "false" This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to "true", then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.
	ID	**ID** specifies a marker under which the hash values calculated go. Multiple infoblock hashes can go under same ID.
	TYPE	**TYPE** specifies the type of the input file specified. For e.g. IHEX, S19,bin file etc.
	MODE	**MODE** specifies the endianness of a file. It may take values BigEndian or LittleEndian.
	OID_PUB	It specifies the Public Key OID value. Based on last digit of this value, hash algorithm is invoked. **BOSCH TRUST CENTER:** 1 – SHA256 2 – SHA256 **VW TRUST CENTER:** 1 – SHA256 2 – SHA256 3 – SHA512
INFOBLOCK	**INFOBLOCKADDR**	**INFOBLOCK** tag specifies the starting address of infoblock link chain or the address of the infoblock which has to be built, which is specified in attribute **INFOBLOCKADDR**.

RANGES	**RANGE, OUTPUT**	**RANGES** tag provides provision to calculate HASH over a range of data.
	OUTPUT	**OUTPUT** attribute specifies the path/name of the input hex file. This is an optional attribute. This attribute provides file name for writing output file. User can also specify the file name along with the path from which HexmodX pick up the files. If path is not specified, HexmodX shall pick up the files from application directory. If **OUTPUT** is not specified in **RANGES**, the hash calculated under **RANGES** will go into **OUTPUT** file of the parent **INPUT** tag.
RANGE	**FROM, TO**	**RANGE** tag is used to specify the extremes of the range via FROM and TO attributes.
	FROM	**FROM** attribute is used to mark the start address of the range of hash calculation.
	TO	**TO** attribute is used to mark the end address of the range of hash calculation.

3.2.3.2 Sign Calculation

This feature is used to generate the signed hex file from the input hex file. Please find below the sample xml file to generate the sign text file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_HEX INFOBLKADDR="0x80020000"
INPUTFILE="medc18.hex" IGNORELINECKS="true" TYPE="IHEX"
KEYPATH="RBAROOT1:CN:DUMMY\tRBA1DGS1:CN:DUMMY\tRBA1DGS1/OEMA1:CN:DUMMY">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX"
LYT="0x20"/>
                  </INPUT_HEX>
            </SIGNATURE>
       </MDG1TPROT>
</HEXMOD>
```

```
</CONF>
```

Elements	Children or Attributes	Description
MDG1TPROT		A tag to specify the Signature operations
	AUTOSIGNFILE	Attribute to specify Auto resign xml file which stores the smart card pin and smart card serial number.
SIGNATURE	**MODE** **INPUT_HEX**	**MODE** tag provides options to perform Signature calculation either “ONLINE” or “OFFLINE”
	KEYPATH	**KEYPATH** attribute is used to define the public/private key information.
INPUT_HEX	**INPUTFILE**	**INPUTFILE** attribute specifies the path/name of the input hex file.
	TYPE	**TYPE** specifies the type of the input file specified. For eg. IHEX or S19 file.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.
	INFOBLKADDR	**INFOBLKADDR** tag specifies the starting address of infoblock link chain or the address of the infoblock which has to be built
	MODE	**MODE** specifies the endianness of a file. It may take values BigEndian or LittleEndian.
	AUTHBITS	User can specify AUTHBITS value.
	SREC_INFOBLK_XML	SREC INFOBLOCK XML attribute specifies the infoblock details of s19 type file. User need to configure start address and end address of all the blocks in S19 file.
OUTPUT	**FILENAME**	**FILENAME** attribute specifies the path/name of the output hex file. This is a mandatory attribute.
	TYPE	**TYPE** specifies the type of the input file specified. For e.g. IHEX or S19 file.

	LYT	**LYT** indicates the layout of the output hex file.
	INFOBLKADDR	**INFOBLKADDR** indicates building infoblock again after patching the signature and certificate.

Sign calculation using user defined authorization bits (AUTHBITS):

Specify authorization bits value in AUTHBITS attribute as below,

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  /mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_HEX INFOBLKADDR="0x80000020"
INPUTFILE="mx18_make_INF.hex" IGNORELINECKS="true" TYPE="IHEX"
AUTHBITS="0,0,0,0,0,0,0,0,0,01,07,255,255,0,0,0,255,255,255,255,255,255,255
,255,255,255,255,255,255,255,255,255">
                        <OUTPUT FILENAME="output1.hex" TYPE="IHEX"
LYT="0x20"/>
                  </INPUT_HEX>
            </SIGNATURE>
      </MDG1TPROT>
<HEXMOD>
</CONF>
```

3.2.3.3 Sign Text Generation

This feature is used to generate the sign text file from the hash text file. Please find below the sample xml to generate the sign text file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE">
                  <INPUT_HASH INPUT="hash.txt" OUTPUT="sign.txt"/>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

To generate signature and certificate values for HSM configuration area, specify HSM xml configuration file in HSM_FILE attribute of SIGNATURE element.

Children or	Description

Attributes	
INPUT_HASH	Calculates sign text file by reading input hash text file.
	KEYPATH attribute is used to define the public/private key information.
	AUTHBITS attribute - User can specify AUTHBITS value.
	INPUT attribute specifies the path/name of the input hash text file for which signatures are to be generated.
	OUTPUT attribute specifies the path/name of the output text file. The output text file contains the signatures for each hash value from the input hash text file.
	OID_PUB - It specifies the Public Key OID value. Based on last digit of this value, hash algorithm is invoked. **BOSCH TRUST CENTER:** 1 – RSA 2 – ECDSA **VW TRUST CENTER:** 1 – RSA 2 – ECDSA

3.2.3.4 Sign Patching

This feature is used to generate the signed hex file by patching the Signature and Certificate values from Signature text file. Please find below the sample xml to generate the sign text file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1    .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
        <SIGNATURE MODE="ONLINE"
KEYPATH="RBAROOT1:CN:DUMMY\tRBA1DGS1:CN:DUMMY\tRBA1DGS1/OEMA1:CN:DUMMY">
                  <INPUT_SIGN INPUTFILE="input1.hex" INPUT_TXT="sign.txt"
IGNORELINECKS="true" TYPE="IHEX" MODE="LittleEndian"
INFOBLKADDR="0x80020000"  ID="Tag1">
                        <OUTPUT FILENAME="output1.hex" TYPE="IHEX"
LYT="0x20"/>
               </INPUT_SIGN>
         </SIGNATURE>
       </MDG1TPROT>
</HEXMOD>
</CONF>
```

To patch signature and certificate values for HSM configuration area, specify HSM xml configuration file in HSM_FILE attribute of SIGNATURE element.

Elements	Children or Attributes	Description
SIGNATURE	**MODE** **TC_TYPE**	**MODE** attribute provides options to perform Signature calculation either “ONLINE” or “OFFLINE”.

		TC_TYPE attribute specifies TRUST CENTER type. It takes the values BOSCH_TRUST_CENTER and VW_TRUST_CENTER. It is optional and default value is BOSCH_TRUST_CENTER.
	HSM_FILE	**HSM_FILE** attribute specifies HSM xml configuration in which signature and certificate address of HSM configuration area is configured for patching the values. It is an optional attribute.
INPUT_SIGN	**INPUT**	**INPUT** attribute specifies the path/name of the input hash text file.
	INPUT_TXT	**INPUT_TXT** attribute specifies the path/name of the input text file which contains the signature and the certificate values to be patched.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found. Application will throw ERROR and abort if this option is not set.
	ID	**ID** specifies a marker from which the signature calculated needs to be extracted for patching.
	MODE	**MODE** specifies the endianness of a file. It may take values BigEndian or LittleEndian.
	INFOBLKADDR	**INFOBLKADDR** tag specifies the starting address of infoblock link chain or the address of the infoblock which has to be built
OUTPUT	**FILENAME**	**FILENAME** attribute specifies the path/name of the output hex file. This is a mandatory attribute.
	TYPE	**TYPE** specifies the type of the input file specified. For eg. IHEX or S19 file.
	LYT	**LYT** indicates the layout of the output hex file.

3.2.3.5 Sign and certificate Calculation with TSW

This feature is used to generate the signed hex file from the input hex file. Note that **TSW** indicates “test software” use case. It does not contain memory blocks with defined ranges.

Please find below the sample xml to generate the sign text file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE" TYPE="TSW"
KEYPATH="RBAROOT1:CN:DUMMY\tRBA1DGS1:CN:DUMMY\tRBA1DGS1/OEMA1:CN:DUMMY">
                  <INPUT_HEX INFOBLKADDR="0x80020000"
INPUTFILE="medc18.hex" IGNORELINECKS="true"  TYPE="IHEX">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX"
LYT="0x20"/>
                  </INPUT_HEX>
            </SIGNATURE>
       </MDG1TPROT>
</HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
MDG1TPROT	**MIDWAREPATH**	**MIDWAREPATH** tag specifies the path of the PKCS11 library DLL.
SIGNATURE	**MODE** **TYPE** **INPUT_HEX**	**MODE** tag provides options to perform Signature calculation either “ONLINE” or “OFFLINE”
	TYPE	**TYPE** attribute is used to specify the TSW mode for signature. If the TSW is specified, then infoblock address is ignored and the signature is calculated for the entire file and patched in the end of the file. Also the password for the PIN is saved within the tool and used for further sign processing. The user will be asked to enter the PIN for the first time only.
	KEYPATH	**KEYPATH** attribute is used to define the public/private key information.
	TC_TYPE	Specifies the trust center type. Value can be BOSCH_TRUST_CENTER and VW_TRUST_CENTER
	SIGNATURE_ALGO	Specifies the signature algorithm type. Value can be RSA and ECDSA.
INPUT_HEX	**INPUTFILE**	**INPUTFILE** attribute specifies the path/name of the input hex file.
	TYPE	**TYPE** specifies the type of the input file specified. For eg. IHEX or S19 file.
	IGNORELINECKS	This is an optional boolean attribute with default value “false” This attribute provides HexmodX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexmodX will issues a WARNING message for the wrong line checksum found.

		Application will throw ERROR and abort if this option is not set.
	MODE	**MODE** specifies the endianness of a file. It may take values BigEndian or LittleEndian.
OUTPUT	**FILENAME**	**FILENAME** attribute specifies the path/name of the output hex file. This is a mandatory attribute.
	TYPE	**TYPE** specifies the type of the input file specified. For eg. IHEX or S19 file.
	LYT	**LYT** indicates the layout of the output hex file.
	INFOBLKADDR	**INFOBLKADDR** indicates building infoblock again after patching the signature and certificate.

Certificate Calculation with TSW:

This feature is used to generate the certificate for the input hex file. Certificate values are appended to the end of the input hex file.

Please find below the sample xml to generate the certificate file.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
        <SIGNATURE MODE="ONLINE" TYPE="TSW"
KEYPATH="RBAROOT1:CN:DUMMY\tRBA1DGS1:CN:DUMMY\tRBA1DGS1/OEMA1:CN:DUMMY">
            <INPUT_HEX INFOBLKADDR="0x80020000" INPUTFILE="medc18.hex"
IGNORELINECKS="true"  TYPE="IHEX">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX"
LYT="0x20"/>
                  </INPUT_HEX>
            </SIGNATURE>

            <CERTIFICATE>
                  <INPUT INFOBLKADDR=" " INPUT="header.hex"
OUTPUT="header_out_cert.hex" IGNORELINECKS="true" LYT="0x10"/>
            </CERTIFICATE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

3.2.3.6 Dummy Signature and certificate calculation

This feature is used to generate a signed hex file with dummy Signature and Certificate values. It is supported both for MEMLAY and TSW usecases.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="OFFLINE" DUMMY="true">
                  <INPUT_HEX INFOBLKADDR="0x08FC0020" INPUTFILE="medc18_v9.hex"
IGNORELINECKS="true" TYPE="IHEX">
                        <OUTPUT FILENAME="output.hex" TYPE="IHEX" LYT="0x20"/>
                  </INPUT_HEX>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

The attribute DUMMY in signature tag is used to generate dummy signature and certificate. If DUMMY=”true”, HexmodX creates dummy signed hex file, false would use middleware to sign the hex file.

To generate a dummy signed text file using hash text file, example is shown below,

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="OFFLINE" DUMMY="true">
              <INPUT_HASH INPUT="hash.txt" OUTPUT="sign.txt"/>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

To patch dummy Signature and Certificate, use the below example,

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="OFFLINE" DUMMY="true">
                  <INPUT_SIGN INPUTFILE="medc18_1.hex" INPUT_TXT="sign.txt"
IGNORELINECKS="true" TYPE="IHEX" MODE="LittleEndian" ID="tag1"
INFOBLKADDR="0x80020000">
                        <OUTPUT FILENAME="output1.hex" TYPE="IHEX" LYT="0x10"/>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

3.2.3.7 Save PIN for multiple runs of MDG1tprot

To avoid entering login password for multiple runs of HexmodX for signing use cases, the following xml configuration can be defined. With respect to below example, smart card pin would be stored in c:\\temp directory within PIN.txt file in an encrypted form. Encryption is done using AES Encryption algorithm.

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  .\schemas\mdg1tprot.xsd">
<HEXMOD>
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE" LASTRUN="0" PINPATH="c:\\temp">
                  <INPUT_HASH INPUT="hash.txt" OUTPUT="sign.txt"
KEYPATH="RBAROOT1:CN:DUMMY\RBA1DGS1:CN:DUMMY\RBA1DGS1/OEMA1:CN:DUMMY"
OID="1.3.6.1.4.1.5318.2504.21.2.1.1.7.0.0"/>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
SIGNATURE	**MODE**	**MODE** tag provides options to perform Signature calculation either “ONLINE” or “OFFLINE”.
	LASTRUN	**LASTRUN** can be 0 or 1 to indicate if the run is last run or not. **0** : There are multiple runs after the present run **1** : Last Run
	PINPATH	**PINPATH** attribute specifies the path to which the encrypted PIN file must be stored. The path will create PIN.txt file containing the PIN. This file will be used to login for all the other runs.

By providing the **LASTRUN** and **PINPATH** attribute within the **SIGNATURE** tag, multiple entries of password can be avoided.

Note that when LASTRUN = 1 i.e. last run of HexmodX, created PIN.txt would be deleted.

3.2.3.8 Derivation of KEYPATH and OID

HexmodX can derive KEYPATH and OID from hex file using the below configuration file,

```
<?xml version="1.0" encoding="utf-8"?>
<CONF xmlns="www.rbin.com/mds-1"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="www.rbin.com/mds-1  /mdg1tprot.xsd">
<HEXMOD>
```

```
      <MDG1TPROT>
            <SIGNATURE MODE="ONLINE">
                  <GENERATE_CERTIFICATE_KEYS INPUTFILE="input.hex" MODE = "BigEndian"
IGNORELINECKS="true" TYPE="IHEX">
                        <OUTPUT FILENAME="SignInfo.xml"/>
                  </GENERATE_CERTIFICATE_KEYS>
            </SIGNATURE>
      </MDG1TPROT>
</HEXMOD>
</CONF>
```

Please note that KEYPATH and OID should be present in hex file. Otherwise HexmodX would serialize empty tag elements in output.xml file.

To generate certificate keys information based on **HSM configuration area** xml file, provide the optional attributes **HSM_FILE** and **HSM_CONFIG** in SIGNATURE element. For example,

<SIGNATURE MODE="ONLINE" HSM_FILE="hsm.xml" HSM_CONFIG="true">

3.2.3.8 Dummy key check

HexmodX can validate if KEYPATH, OID, Public key and Symmetric keys contains dummy values. If it contains, HexmodX throws warning in HexmodX log file.